zip and enumerate iterators

Generated by Doxygen 1.9.8

1 zip and enumerate iterators 1

1.1 Python-like Zip and Enumerate Iterators . . . 1

1.1.1 Properties . . . 1

1.1.2 Code Examples . . . 1

1.1.3 Doxygen Documentation . . . 2

2 Namespace Documentation 2

2.1 iterators Namespace Reference . . . 2

2.1.1 Detailed Description . . . 3

2.1.2 Function Documentation . . . 3

2.2 iterators::impl Namespace Reference . . . 7

2.2.1 Detailed Description . . . 8

2.2.2 Function Documentation . . . 8

2.3 iterators::impl::traits Namespace Reference . . . 10

2.3.1 Detailed Description . . . 11

3 Class Documentation 11

3.1 iterators::impl::CounterIterator< T > Struct Template Reference . . . 11

3.1.1 Detailed Description . . . 12

3.1.2 Constructor & Destructor Documentation . . . 13

3.1.3 Member Function Documentation . . . 13

3.1.4 Friends And Related Symbol Documentation . . . 19

3.2 iterators::impl::CounterRange< T > Struct Template Reference . . . 19

3.2.1 Detailed Description . . . 20

3.2.2 Constructor & Destructor Documentation . . . 20

3.2.3 Member Function Documentation . . . 20

3.3 iterators::impl::RefTuple< Ts > Struct Template Reference . . . 21

3.3.1 Detailed Description . . . 22

3.4 iterators::impl::SynthesizedOperators< Impl > Struct Template Reference . . . 22

3.4.1 Detailed Description . . . 23

3.4.2 Member Function Documentation . . . 24

3.4.3 Friends And Related Symbol Documentation . . . 27

3.5 iterators::impl::Unreachable Struct Reference . . . 28

3.5.1 Detailed Description . . . 28

3.6 iterators::impl::ZipIterator< Iterators > Class Template Reference . . . 29

3.6.1 Detailed Description . . . 30

3.6.2 Member Function Documentation . . . 31

3.6.3 Friends And Related Symbol Documentation . . . 38

4 File Documentation 38

4.1 Iterators.hpp File Reference . . . 38

4.1.1 Detailed Description . . . 40

4.1.2 Macro Definition Documentation . . . 41

4.2 Iterators.hpp 42

Index 59

1 zip and enumerate iterators

1.1 Python-like Zip and Enumerate Iterators

C++-implementation of Python-like zip- and enumerate-iterators which can be used in range-based for loops along with structured bindings to iterate over multiple containers at the same time. Requires C++17.

The library has been tested on the following compilers:

- g++:
  - C++17 support: g++-9 to g++-12
  - C++20 ranges compatibility: g++-10 to g++-12
- clang:
  - C++17 support: clang-9 to clang-17
  - C++20 ranges compatibility: clang-16 and clang-17

1.1.1 Properties

The `zip`-class is a container-wrapper for arbitrary iterable containers. It provides the member functions `begin()` and `end()` enabling it to be used in range-based for loops to iterate over multiple containers at the same time. The `enuerate`-function is a special case of `zip` and uses a "counting container" (similar to `std::ranges::iota`) to provide an index. Additionally, const-versions exist which do not allow the manipulation of the container elements.

1.1.2 Code Examples

The syntax is mostly similar to Python:

```
#include <vector>
#include <list>
#include "Iterators.hpp"

using namespace iterators;
std::list<std::string> strings{"a", "b", "c"};
std::vector<int> numbers{1, 2, 3};
for (auto [string, number] : zip(strings, numbers)) {
    // 'string' and 'number' are references to the container element
    string += std::to_string(number);
}

// now 'strings' contains {"a1", "b2", "c3"}
```

The for loop uses so called `ZipIterator`s which point to tuples which in turn contain references to the container elements. Therefore, no copying occurs and manipulation of the container elements is possible. Observe that the structured binding captures by value (since the values are themselves references).

If you want to prohibit manipulation, you can use `const_zip`

```
using namespace iterators;
```

```
std::list<std::string> strings{"a", "b", "c"};
std::vector<int> numbers{1, 2, 3};
for (auto [string, number] : const_zip(strings, numbers)) {
    // string += std::to_string(number);  error, string is readonly!
    std::cout « string « " " « number « std::endl;
}
```

Additionally, you can use `zip_i` to manually zip iterators or pointers:

```
using namespace iterators;
std::list<std::string> strings{"a", "b", "c"};
std::vector<int> numbers{1, 2, 3};
auto zipBegin = zip_i(strings.begin(), numbers.begin());
auto zipEnd = zip_i(strings.end(), numbers.end());
while (zipBegin != zipEnd) {
    auto [s, num] = *zipBegin;
    // ...
    ++zipBegin;
}
```

`ZipIterators` support the same operations as the least powerful underlying iterator. For example, if you zip a random access iterator (e.g. from `std::vector`) and a bidirectional iterator (e.g. from `std::list`), then the resulting `ZipIterator` will only support bidirectional iteration but no random access.

As in Python, the shortest range decides the overall range:

```
using namespace iterators;
std::list<std::string> strings{"a", "b", "c"};
std::vector<int> numbers{1, 2, 3, 4, 5, 6};
for (auto [string, number] : zip(strings, numbers)) {
    std::cout « string « " " « number « " | "
}
// prints a 1 | b 2 | c 3 |
```

The `enumerate`-function works similarly.

```
using namespace iterators;
std::list<std::string> strings{"a", "b", "c"};
for (auto [index, string] : enumerate(strings)) {
    string += std::to_string(index);
}
// now 'strings' contains {"a0", "b1", "c2"}
```

Also, an optional offset can be specified:

```
for (auto [index, string] : enumerate(strings, 4)) { // index starts from 4
    ...
}
```

And as with `zip`, a const version (`const_enumerate`) exists.

In case temporary containers are used, `zip` and `enumerate` will take ownership of the containers to guarantee well-defined memory access.

```
for (auto [index, number] : enumerate(std::array{53, 21, 17})) {
    // enumerate takes ownership of the array. The elements
    // can safely be accessed and manipulated
}
```

1.1.3 Doxygen Documentation

- HTML
- PDF

2 Namespace Documentation

2.1 iterators Namespace Reference

namespace containing zip and enumerate functions

Namespaces

- namespace impl

  namespace containing structures and helpers used to implement zip and enumerate. Normally there is no need to use any of its members directly

Functions

- template<typename ... Iterators>
  constexpr auto zip_i (Iterators ...iterators) -> impl::ZipIterator< std::tuple< Iterators... > >
- template<typename ... Iterable>
  constexpr auto zip (Iterable &&...iterable)
- template<typename ... Iterable>
  constexpr auto const_zip (Iterable &&...iterable)
- template<typename Container , typename T = std::size_t>
  constexpr auto enumerate (Container &&container, T start=T(0), T increment=T(1))
- template<typename Container , typename T = std::size_t>
  constexpr auto const_enumerate (Container &&container, T start=T(0), T increment=T(1))
- template<typename ... Iterable>
  constexpr auto zip_enumerate (Iterable &&...iterable)
- template<typename ... Iterable>
  constexpr auto const_zip_enumerate (Iterable &&...iterable)

2.1.1 Detailed Description

namespace containing zip and enumerate functions

2.1.2 Function Documentation

const_enumerate()

```
template<typename Container , typename T = std::size_t>
constexpr auto iterators::const_enumerate (
            Container && container,
            T start = T(0),
            T increment = T(1) )  [constexpr]
```

enumerate variant that does not allow manipulation of the container elements

Function that can be used in range based loops to emulate the enumerate iterator from python.

Template Parameters

Container	Container type that supports iteration
T	type of enumerate counter (default std::size_t)

Parameters

container	Source container

Parameters

start	Optional index offset (default 0)
increment	Optional index increment (default 1)

Returns

impl::ZipView that provides begin and end members to be used in range based for-loops.

const_zip()

```
template<typename ...  Iterable>
constexpr auto iterators::const_zip (
            Iterable &&...  iterable )  [constexpr]
```

Zip variant that does not allow manipulation of the container elements

Function that can be used in range based loops to emulate the zip iterator from python. As in python: if the passed containers have different lengths, the container with the least items decides the overall range

Template Parameters

Iterable	Container types that support iteration

Parameters

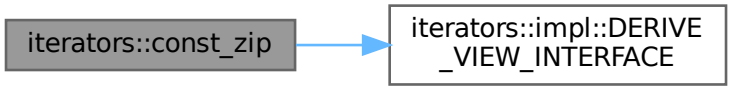
iterable	Arbitrary number of containers

Returns

impl::ZipView class that provides begin and end members to be used in range based for-loops

References iterators::impl::DERIVE_VIEW_INTERFACE().

Here is the call graph for this function:



const_zip_enumerate()

```
template<typename ...  Iterable>
constexpr auto iterators::const_zip_enumerate (
             Iterable &&...  iterable )  [constexpr]
```

zip_enumerate variant that does not allow manipulation of the container elements

combination of zip and enumerate, i.e. returns an impl::ZipView that contains an enumerator at the first position

Template Parameters

Iterable	Types of arguments

Parameters

iterable	arbitrary number of iterables followed by optionally a start and an increment

Returns

impl::ZipView with prepended enumerator

enumerate()

```
template<typename Container , typename T = std::size_t>
constexpr auto iterators::enumerate (
             Container && container,
             T start = T(0),
             T increment = T(1) )  [constexpr]
```

Function that can be used in range based loops to emulate the enumerate iterator from python.

Template Parameters

Container	Container type that supports iteration
T	type of enumerate counter (default std::size_t)

Parameters

container	Source container
start	Optional index offset (default 0)
increment	Optional index increment (default 1)

Returns

impl::ZipView that provides begin and end members to be used in range based for-loops.

zip()

```
template<typename ...  Iterable>
constexpr auto iterators::zip (
            Iterable &&...  iterable )  [constexpr]
```

Function that can be used in range based loops to emulate the zip iterator from python. As in python: if the passed containers have different lengths, the container with the least items decides the overall range

Template Parameters

Iterable	Container types that support iteration

Parameters

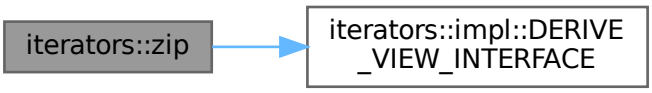
iterable	Arbitrary number of containers

Returns

impl::ZipView class that provides begin and end members to be used in range based for-loops

References iterators::impl::DERIVE_VIEW_INTERFACE().

Here is the call graph for this function:



zip_enumerate()

```
template<typename ...  Iterable>
constexpr auto iterators::zip_enumerate (
            Iterable &&...  iterable )  [constexpr]
```

combination of zip and enumerate, i.e. returns an impl::ZipView that contains an enumerator at the first position

Template Parameters

Iterable	Types of arguments

Parameters

iterable	arbitrary number of iterables followed by optionally a start and an increment

Returns

impl::ZipView with prepended enumerator

zip_i()

```
template<typename ...  Iterators>
constexpr auto iterators::zip_i (
            Iterators ...  iterators ) -> impl::ZipIterator<std::tuple<Iterators...>>   [constexpr]
```

Function that is used to create a impl::ZipIterator from an arbitrary number of iterators

Template Parameters

Iterators	type of iterators

Parameters

iterators	arbitrary number of iterators

Returns

impl::ZipIterator

Note

ZipIterators have the same iterator category as the least powerful underlying operator. This means that for example, zipping a random access iterator and a bidirectional iterator only yields a bidirectional impl::ZipIterator

2.2 iterators::impl Namespace Reference

namespace containing structures and helpers used to implement zip and enumerate. Normally there is no need to use any of its members directly

Namespaces

- namespace traits

  namespace containing type traits used in implementation of zip and enumerate

Classes

- struct CounterIterator

  Iterator of an infinite sequence of numbers. Simply increments an internal counter.
- struct CounterRange

  Represents an infinite range of numbers.
- struct RefTuple

  Proxy reference type that supports assignment and swap even on const instances.
- struct SynthesizedOperators

  CRTP-class that provides additional pointer arithmetic operators synthesized from basic operators.
- struct Unreachable

  represents the unreachable end of an infinite sequence
- class ZipIterator

  Class combining multiple iterators into one. Use it to iterate over multiple ranges at the same time.

Functions

- template<std::size_t Offset, std::size_t ... Idx>
  constexpr auto **index_seq_impl** (std::index_sequence< Idx... >) noexcept
- template<std::size_t Start, std::size_t End>
  constexpr auto **index_seq** () noexcept
- template<typename Tuple , std::size_t ... Idx1, std::size_t ... Idx2>
  constexpr auto **tuple_split_impl** (Tuple &&tuple, std::index_sequence< Idx1... >, std::index_sequence< Idx2... >)
- template<std::size_t Idx, typename Tuple >
  constexpr auto **tuple_split** (Tuple &&tuple)
- template<template< typename > typename Predicate, std::size_t Idx, typename ... Ts>
  constexpr std::size_t **index_of** () noexcept
- template<typename Tuple >
  constexpr auto **swap** (Tuple &&a, Tuple &&b) -> std::enable_if_t< iterators::impl::traits::is_same_template_v< iterators::impl::RefTuple, Tuple > >
- template<typename ... Iterable>
  struct ZipView DERIVE_VIEW_INTERFACE (ZipView< Iterable... >)

  Zip-view that provides begin() and end() member functions. Use to loop over multiple ranges at the same time using ranged based for-loops.
- template<typename T >
  constexpr T sgn (T val) noexcept

2.2.1 Detailed Description

namespace containing structures and helpers used to implement zip and enumerate. Normally there is no need to use any of its members directly

2.2.2 Function Documentation

DERIVE_VIEW_INTERFACE()

```
template<typename ...  Iterable>
struct ZipView iterators::impl::DERIVE_VIEW_INTERFACE (
            ZipView< Iterable...  >  )
```

Zip-view that provides begin() and end() member functions. Use to loop over multiple ranges at the same time using ranged based for-loops.

Ranges are captured by lvalue reference, no copying occurs. Temporaries are allowed as well in which case storage is moved into the zip-view.

Template Parameters

Iterable	Underlying range types

CTor. Binds reference to ranges or takes ownership in case of rvalue references

Template Parameters

Container	range types

Parameters

containers	arbitrary number of ranges

Returns a ZipIterator to the first elements of the underlying ranges

Returns

ZipIterator created by invoking std::begin on all underlying ranges

Returns a ZipIterator to the elements following the last elements of the the underlying ranges

Returns

ZipIterator created by invoking std::end on all underlying ranges

Note

returns a ZipIterator that does not allow changing the ranges' elements

Array subscript operator (no bounds are checked)

Template Parameters

IsRandomAccess	SFINAE helper, do not specify explicitly

Parameters

index	index

Returns

zip view element at given index

Returns the smallest size of all containers. Only available if all containers know their size

Template Parameters

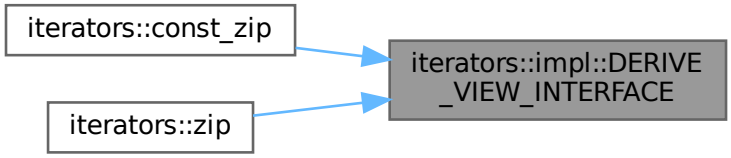
HasSize	SFINAE guard, do not specify explicitly

Returns

smallest size of all containers

Referenced by iterators::const_zip(), and iterators::zip().

Here is the caller graph for this function:



sgn()

```
template<typename T >
constexpr T iterators::impl::sgn (
            T val )  [constexpr], [noexcept]
```

Signum function

Template Parameters

T	arbitrary scalar type

Parameters

val	function argument

Returns

+1 if val >= 0, -1 else

2.3 iterators::impl::traits Namespace Reference

namespace containing type traits used in implementation of zip and enumerate

2.3.1 Detailed Description

namespace containing type traits used in implementation of zip and enumerate

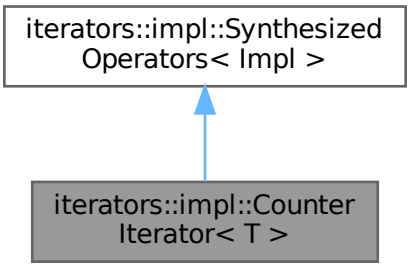
3 Class Documentation

3.1 iterators::impl::CounterIterator< T > Struct Template Reference

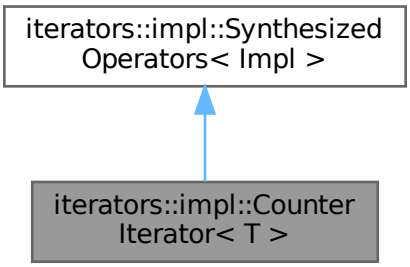
Iterator of an infinite sequence of numbers. Simply increments an internal counter.

`#include <Iterators.hpp>`

Inheritance diagram for iterators::impl::CounterIterator< T >:



Collaboration diagram for iterators::impl::CounterIterator< T >:



Public Types

- using **value_type** = T
- using **reference** = T
- using **pointer** = void
- using **iterator_category** = std::random_access_iterator_tag
- using **difference_type** = std::ptrdiff_t

Public Member Functions

- constexpr CounterIterator (T begin, T increment=T(1)) noexcept
- constexpr CounterIterator & operator++ () noexcept
- constexpr CounterIterator & operator-- () noexcept
- constexpr CounterIterator & operator+= (difference_type n) noexcept
- constexpr CounterIterator & operator-= (difference_type n) noexcept
- constexpr difference_type operator- (const CounterIterator &other) const noexcept
- constexpr bool operator== (const CounterIterator &other) const noexcept
- constexpr bool operator< (const CounterIterator &other) const noexcept
- constexpr bool operator> (const CounterIterator &other) const noexcept
- constexpr T operator* () const noexcept
- template<REQUIRES_IMPL(Impl, *(INSTANCE_OF_IMPL+INSTANCE_OF(typename Implementation::difference_type))) > constexpr decltype(auto) operator[ ] (typename Implementation::difference_type n) const noexcept(noexcept(*(std::declval< Impl >()+n)))
- template<REQUIRES_IMPL(Impl, *(INSTANCE_OF_IMPL+INSTANCE_OF(typename Implementation::difference_type))) > constexpr decltype(auto) operator[ ] (typename Implementation::difference_type n) noexcept(noexcept(*(std::declval< Impl >()+n)))
- template<REQUIRES_IMPL(Impl,++INSTANCE_OF_IMPL) > constexpr Impl operator++ (int) noexcept(noexcept(++std::declval< Impl >()) &&std::is_nothrow_copy_constructible_v< Impl >)
- template<REQUIRES_IMPL(Impl, --INSTANCE_OF_IMPL) > constexpr Impl operator-- (int) noexcept(noexcept(--std::declval< Impl >()) &&std::is_nothrow_copy_constructible_v< Impl >)
- template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL==INSTANCE_OF(T)) > constexpr bool operator!= (const T &other) const noexcept(noexcept(INSTANCE_OF_IMPL==other))
- template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL > INSTANCE_OF(T)) > constexpr bool operator<= (const T &rhs) const noexcept(noexcept(INSTANCE_OF_IMPL > rhs))
- template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL< INSTANCE_OF(T)) > constexpr bool operator>= (const T &rhs) const noexcept(noexcept(INSTANCE_OF_IMPL< rhs))

Friends

- constexpr bool operator== (const CounterIterator &, Unreachable) noexcept
- constexpr bool operator== (Unreachable, const CounterIterator &) noexcept
- constexpr bool **operator!=** (Unreachable, const CounterIterator &) noexcept

3.1.1 Detailed Description

template<typename T = std::size_t>
struct iterators::impl::CounterIterator< T >

Iterator of an infinite sequence of numbers. Simply increments an internal counter.

Template Parameters

Type	of the counter (most of the time this is `std::size_t`)

3.1.2 Constructor & Destructor Documentation

CounterIterator()

```
template<typename T = std::size_t>
constexpr iterators::impl::CounterIterator< T >::CounterIterator (
            T begin,
            T increment = T(1) )  [inline], [explicit], [constexpr], [noexcept]
```

CTor.

Parameters

begin	start of number sequence
increment	step size (default is 1)

Note

Depending on the template type T, increment can also be negative.

3.1.3 Member Function Documentation

operator"!=()

```
template<typename Impl >
template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL==INSTANCE_OF(T)) >
constexpr bool iterators::impl::SynthesizedOperators< Impl >::operator!= (
            const T & other ) const  [inline], [constexpr], [noexcept], [inherited]
```

Inequality comparison

Template Parameters

T	type of right hand side
Implementation	SFINAE helper, do not specify explicitly

Parameters

other	right hand side

Returns

true if this is not equal to other

operator∗()

```
template<typename T = std::size_t>
constexpr T iterators::impl::CounterIterator< T >::operator* ( ) const  [inline], [constexpr],
[noexcept]
```

Produces the counter value

Returns

value of internal counter

operator++() [1/2]

```
template<typename T = std::size_t>
constexpr CounterIterator & iterators::impl::CounterIterator< T >::operator++ ( )  [inline],
[constexpr], [noexcept]
```

Increments value by increment

Returns

reference to this

operator++() [2/2]

```
template<typename Impl >
template<REQUIRES_IMPL(Impl,++INSTANCE_OF_IMPL) >
constexpr Impl iterators::impl::SynthesizedOperators< Impl >::operator++ (
            int  )  [inline], [constexpr], [noexcept], [inherited]
```

Postfix increment. Synthesized from prefix increment

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Returns

Instance of Impl

operator+=()

```
template<typename T = std::size_t>
constexpr CounterIterator & iterators::impl::CounterIterator< T >::operator+= (
            difference_type n )  [inline], [constexpr], [noexcept]
```

Compound assignment increment. Increments value by n times increment

Parameters

n	number of steps

Returns

reference to this

operator-()

```
template<typename T = std::size_t>
constexpr difference_type iterators::impl::CounterIterator< T >::operator- (
            const CounterIterator< T > & other ) const  [inline], [constexpr], [noexcept]
```

Difference between two CounterIterators

Parameters

other	right hand side

Returns

integer `n` with the smallest possible absolute value such that `other + n <= *this`

Note

When other has the same increment as `*this`, then the returned value is guaranteed to fulfil `other + n == *this`. In the following example, this is not the case:

```
CounterIterator a(8, 1);
CounterIterator b(4, 3);
auto diff = a - b; // yields 1 since b + 1 <= a
```

operator--() [1/2]

```
template<typename T = std::size_t>
constexpr CounterIterator & iterators::impl::CounterIterator< T >::operator-- ( )  [inline],
[constexpr], [noexcept]
```

Decrements value by increment

Returns

reference to this

operator--() [2/2]

```
template<typename Impl >
template<REQUIRES_IMPL(Impl, --INSTANCE_OF_IMPL) >
constexpr Impl iterators::impl::SynthesizedOperators< Impl >::operator-- (
            int  )  [inline], [constexpr], [noexcept], [inherited]
```

Postfix decrement. Synthesized from prefix decrement

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Returns

Instance of Impl

operator-=()

```
template<typename T = std::size_t>
constexpr CounterIterator & iterators::impl::CounterIterator< T >::operator-= (
            difference_type n )  [inline], [constexpr], [noexcept]
```

Compound assignment decrement. Increments value by n times increment

Parameters

n	number of steps

Returns

reference to this

operator<()

```
template<typename T = std::size_t>
constexpr bool iterators::impl::CounterIterator< T >::operator< (
            const CounterIterator< T > & other ) const  [inline], [constexpr], [noexcept]
```

Less comparison of internal counters with respect to increment of this instance

Parameters

other	right hand side

Returns

true if
`sgn(increment) **this < *other sgn(increment)`
where `sgn` is the signum function

Note

If increment is negative then both sides of the inequality are multiplied with -1. For example: let `it1 = 5` and `it2 = -2` be two CounterIterators where `it1` has negative increment. Then `it1 < it2` is true.

operator<=()

```
template<typename Impl >
template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL > INSTANCE_OF(T)) >
constexpr bool iterators::impl::SynthesizedOperators< Impl >::operator<= (
            const T & rhs ) const  [inline], [constexpr], [noexcept], [inherited]
```

Less than or equal comparison

Template Parameters

T	type of right hand side
Implementation	SFINAE helper, do not specify explicitly

Parameters

other	right hand side

Returns

true if this is not greater than other

operator==()

```
template<typename T = std::size_t>
constexpr bool iterators::impl::CounterIterator< T >::operator== (
            const CounterIterator< T > & other ) const  [inline], [constexpr], [noexcept]
```

Equality comparison.

Parameters

other	right hand side

Returns

true if counter of left and right hand side are equal

operator>()

```
template<typename T = std::size_t>
constexpr bool iterators::impl::CounterIterator< T >::operator> (
            const CounterIterator< T > & other ) const  [inline], [constexpr], [noexcept]
```

Greater comparison of internal counters with respect to increment of this instance

Parameters

other	right hand side

Returns

true if
`sgn(increment) **this > *other sgn(increment)`

where `sgn` is the signum

Note

If increment is negative then both sides of the inequality are multiplied with -1. For example: let `it1 = 5` and `it2 = -2` be two CounterIterators where `it1` has negative increment. Then `it1 > it2` is false.

operator>=()

```
template<typename Impl >
template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL< INSTANCE_OF(T)) >
constexpr bool iterators::impl::SynthesizedOperators< Impl >::operator>= (
            const T & rhs ) const  [inline], [constexpr], [noexcept], [inherited]
```

Greater than or equal comparison

Template Parameters

T	type of right hand side
Implementation	SFINAE helper, do not specify explicitly

Parameters

other	right hand side

Returns

true if this is not less than other

operator[]() [1/2]

```
template<typename Impl >
template<REQUIRES_IMPL(Impl, *(INSTANCE_OF_IMPL+INSTANCE_OF(typename Implementation::difference↩
_type))) >
constexpr decltype(auto) iterators::impl::SynthesizedOperators< Impl >::operator[] (
            typename Implementation::difference_type n ) const  [inline], [constexpr], [noexcept],
[inherited]
```

Array subscript operator

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Parameters

n	index

Returns

*(*this + n)

operator[]() [2/2]

```
template<typename Impl >
template<REQUIRES_IMPL(Impl, *(INSTANCE_OF_IMPL+INSTANCE_OF(typename Implementation::difference↩
_type))) >
constexpr decltype(auto) iterators::impl::SynthesizedOperators< Impl >::operator[] (
            typename Implementation::difference_type n )  [inline], [constexpr], [noexcept],
[inherited]
```

3.1.4 Friends And Related Symbol Documentation

operator== [1/2]

```
template<typename T = std::size_t>
constexpr bool operator== (
            const CounterIterator< T > & ,
            Unreachable  )  [friend]
```

Equality comparison with Unreachable sentinel

Returns

false

operator== [2/2]

```
template<typename T = std::size_t>
constexpr bool operator== (
            Unreachable ,
            const CounterIterator< T > &  )  [friend]
```

Equality comparison with Unreachable sentinel

Returns

false

The documentation for this struct was generated from the following file:

- Iterators.hpp

3.2 iterators::impl::CounterRange< T > Struct Template Reference

Represents an infinite range of numbers.

```
#include <Iterators.hpp>
```

Public Member Functions

- constexpr CounterRange (T start=T(0), T increment=T(1)) noexcept
- constexpr CounterIterator< T > begin () const noexcept

Static Public Member Functions

- static constexpr Unreachable end () noexcept

3.2.1 Detailed Description

template<typename T = std::size_t>
struct iterators::impl::CounterRange< T >

Represents an infinite range of numbers.

Template Parameters

T	type of number range

3.2.2 Constructor & Destructor Documentation

CounterRange()

```
template<typename T = std::size_t>
constexpr iterators::impl::CounterRange< T >::CounterRange (
            T start = T(0),
            T increment = T(1) )  [inline], [explicit], [constexpr], [noexcept]
```

CTor

Parameters

start	start of the range
increment	step size

Note

Depending on the template type T, increment can also be negative.

3.2.3 Member Function Documentation

begin()

```
template<typename T = std::size_t>
constexpr CounterIterator< T > iterators::impl::CounterRange< T >::begin ( ) const  [inline],
[constexpr], [noexcept]
```

Returns

CounterIterator representing the beginning of the sequence

end()

```
template<typename T = std::size_t>
static constexpr Unreachable iterators::impl::CounterRange< T >::end ( )  [inline], [static],
[constexpr], [noexcept]
```

Returns

Sentinel object representing the unreachable end of the sequence

The documentation for this struct was generated from the following file:

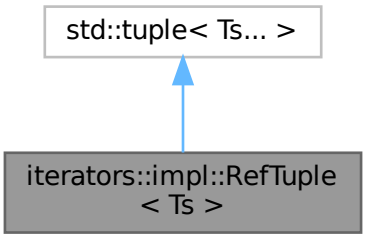
- Iterators.hpp

3.3 iterators::impl::RefTuple< Ts > Struct Template Reference

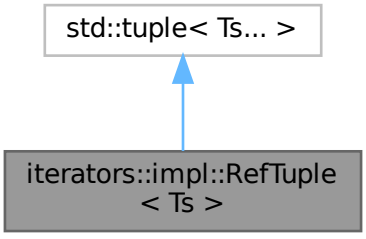
Proxy reference type that supports assignment and swap even on const instances.

`#include <Iterators.hpp>`

Inheritance diagram for iterators::impl::RefTuple< Ts >:



Collaboration diagram for iterators::impl::RefTuple< Ts >:



Public Member Functions

- template<typename Dummy = int>
  constexpr auto **operator=** (RefTuple &&other) noexcept((std::is_nothrow_move_assignable_v< std::remove_reference_t< Ts > > &&...)) -> std::enable_if_t< std::is_same_v< Dummy, int > and Assignable, RefTuple & >
- template<typename ... Us>
  decltype(auto) **operator=** (const std::tuple< Us... > &other) const
- template<typename ... Us>
  decltype(auto) **operator=** (std::tuple< Us... > &&other) const
- template<std::size_t Idx>
  constexpr decltype(auto) **get** () const &noexcept
- template<std::size_t Idx>
  constexpr decltype(auto) **get** () &noexcept
- template<std::size_t Idx>
  constexpr decltype(auto) **get** () const &&noexcept
- template<std::size_t Idx>
  constexpr decltype(auto) **get** () &&noexcept
- template<typename Tuple >
  constexpr auto **swap** (Tuple &&other) const -> std::enable_if_t< std::is_same_v< Tuple, RefTuple > and Assignable >

Static Public Attributes

- static constexpr bool **Assignable** = (not std::is_const_v<std::remove_reference_t<Ts>> && ...)

3.3.1 Detailed Description

template<typename ... Ts>
struct iterators::impl::RefTuple< Ts >

Proxy reference type that supports assignment and swap even on const instances.

Actual constness is determined by element constness

Template Parameters

Ts	

The documentation for this struct was generated from the following file:

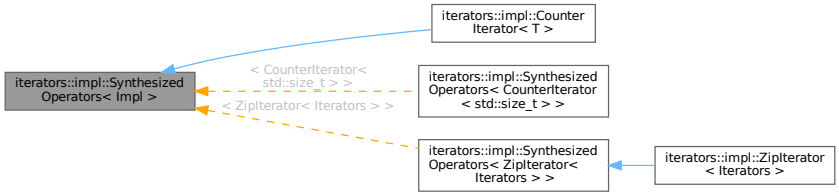
- Iterators.hpp

3.4 iterators::impl::SynthesizedOperators< Impl > Struct Template Reference

CRTP-class that provides additional pointer arithmetic operators synthesized from basic operators.

`#include <Iterators.hpp>`

Inheritance diagram for iterators::impl::SynthesizedOperators< Impl >:



Public Member Functions

- template<REQUIRES_IMPL(Impl, *(INSTANCE_OF_IMPL+INSTANCE_OF(typename Implementation::difference_type))) >
  constexpr decltype(auto) operator[ ] (typename Implementation::difference_type n) const noexcept(noexcept(*(std↩
  ::declval< Impl >()+n)))
- template<REQUIRES_IMPL(Impl, *(INSTANCE_OF_IMPL+INSTANCE_OF(typename Implementation::difference_type))) >
  constexpr decltype(auto) operator[ ] (typename Implementation::difference_type n) noexcept(noexcept(*(std↩
  ::declval< Impl >()+n)))
- template<REQUIRES_IMPL(Impl,++INSTANCE_OF_IMPL) >
  constexpr Impl operator++ (int) noexcept(noexcept(++std::declval< Impl >()) &&std::is_nothrow_copy_↩
  constructible_v< Impl >)
- template<REQUIRES_IMPL(Impl, --INSTANCE_OF_IMPL) >
  constexpr Impl operator-- (int) noexcept(noexcept(--std::declval< Impl >()) &&std::is_nothrow_copy_↩
  constructible_v< Impl >)
- template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL==INSTANCE_OF(T)) >
  constexpr bool operator!= (const T &other) const noexcept(noexcept(INSTANCE_OF_IMPL==other))
- template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL > INSTANCE_OF(T)) >
  constexpr bool operator<= (const T &rhs) const noexcept(noexcept(INSTANCE_OF_IMPL > rhs))
- template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL< INSTANCE_OF(T)) >
  constexpr bool operator>= (const T &rhs) const noexcept(noexcept(INSTANCE_OF_IMPL< rhs))

Friends

- template<REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL+=INSTANCE_OF(typename Implementation::difference_type)) >
  constexpr auto operator+ (Impl it, typename Implementation::difference_type n) noexcept(noexcept(std↩
  ::declval< Impl >()+=n))
- template<REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL+=INSTANCE_OF(typename Implementation::difference_type)) >
  constexpr auto operator+ (typename Implementation::difference_type n, Impl it) noexcept(noexcept(std↩
  ::declval< Impl >()+=n))
- template<REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL -=INSTANCE_OF(typename Implementation::difference_type)) >
  constexpr auto operator- (Impl it, typename Implementation::difference_type n) noexcept(noexcept(std↩
  ::declval< Impl >() -=n))

3.4.1 Detailed Description

template<typename Impl>
struct iterators::impl::SynthesizedOperators< Impl >

CRTP-class that provides additional pointer arithmetic operators synthesized from basic operators.

Adds the following operators

- postfix increment and decrement (requires the respective prefix operators)
- array subscript operator[] (requires operator+ and dereference operator)
- binary arithmetic operators (requires compound assignment operators)
- inequality comparison (requires operator==)
- less than or equal comparison (requires operator>)
- greater than or equal comparison (requires operator<)

Template Parameters

Impl	Base class

3.4.2 Member Function Documentation

operator"!=()

```
template<typename Impl >
template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL==INSTANCE_OF(T)) >
constexpr bool iterators::impl::SynthesizedOperators< Impl >::operator!= (
            const T & other ) const  [inline], [constexpr], [noexcept]
```

Inequality comparison

Template Parameters

T	type of right hand side
Implementation	SFINAE helper, do not specify explicitly

Parameters

other	right hand side

Returns

true if this is not equal to other

operator++()

```
template<typename Impl >
template<REQUIRES_IMPL(Impl,++INSTANCE_OF_IMPL) >
constexpr Impl iterators::impl::SynthesizedOperators< Impl >::operator++ (
            int  )  [inline], [constexpr], [noexcept]
```

Postfix increment. Synthesized from prefix increment

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Returns

Instance of Impl

operator--()

```
template<typename Impl >
template<REQUIRES_IMPL(Impl, --INSTANCE_OF_IMPL) >
constexpr Impl iterators::impl::SynthesizedOperators< Impl >::operator-- (
            int  )  [inline], [constexpr], [noexcept]
```

Postfix decrement. Synthesized from prefix decrement

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Returns

Instance of Impl

operator<=()

```
template<typename Impl >
template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL > INSTANCE_OF(T)) >
constexpr bool iterators::impl::SynthesizedOperators< Impl >::operator<= (
            const T & rhs ) const  [inline], [constexpr], [noexcept]
```

Less than or equal comparison

Template Parameters

T	type of right hand side
Implementation	SFINAE helper, do not specify explicitly

Parameters

other	right hand side

Returns

true if this is not greater than other

operator>=()

```
template<typename Impl >
template<typename T , REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL< INSTANCE_OF(T)) >
constexpr bool iterators::impl::SynthesizedOperators< Impl >::operator>= (
            const T & rhs ) const  [inline], [constexpr], [noexcept]
```

Greater than or equal comparison

Template Parameters

T	type of right hand side
Implementation	SFINAE helper, do not specify explicitly

Parameters

other	right hand side

Returns

true if this is not less than other

operator[]() [1/2]

```
template<typename Impl >
template<REQUIRES_IMPL(Impl, *(INSTANCE_OF_IMPL+INSTANCE_OF(typename Implementation::difference↩
_type))) >
constexpr decltype(auto) iterators::impl::SynthesizedOperators< Impl >::operator[] (
            typename Implementation::difference_type n ) const  [inline], [constexpr], [noexcept]
```

Array subscript operator

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Parameters

n	index

Returns

∗(∗this + n)

operator[]() [2/2]

```
template<typename Impl >
template<REQUIRES_IMPL(Impl, *(INSTANCE_OF_IMPL+INSTANCE_OF(typename Implementation::difference↩
_type))) >
```

```
constexpr decltype(auto) iterators::impl::SynthesizedOperators< Impl >::operator[] (
            typename Implementation::difference_type n )  [inline], [constexpr], [noexcept]
```

3.4.3 Friends And Related Symbol Documentation

operator+ [1/2]

```
template<typename Impl >
template<REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL+=INSTANCE_OF(typename Implementation::difference↩
_type)) >
constexpr auto operator+ (
            Impl it,
            typename Implementation::difference_type n )  [friend]
```

Binary +plus operator. Synthesized from compound assignment operator+=

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Parameters

it	left hand side
n	right hand side

Returns

Instance of Impl

operator+ [2/2]

```
template<typename Impl >
template<REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL+=INSTANCE_OF(typename Implementation::difference↩
_type)) >
constexpr auto operator+ (
            typename Implementation::difference_type n,
            Impl it )  [friend]
```

Binary +plus operator. Synthesized from compound assignment operator+=

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Parameters

n	left hand side
it	right hand side

Returns

Instance of Impl

operator-

```
template<typename Impl >
template<REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL -=INSTANCE_OF(typename Implementation::difference↩
_type)) >
constexpr auto operator- (
            Impl it,
            typename Implementation::difference_type n )  [friend]
```

Binary minus operator. Synthesized from compound assignment operator-=

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Parameters

it	left hand side
n	right hand side

Returns

Instance of Impl

The documentation for this struct was generated from the following file:

- Iterators.hpp

3.5 iterators::impl::Unreachable Struct Reference

represents the unreachable end of an infinite sequence

`#include <Iterators.hpp>`

3.5.1 Detailed Description

represents the unreachable end of an infinite sequence

The documentation for this struct was generated from the following file:

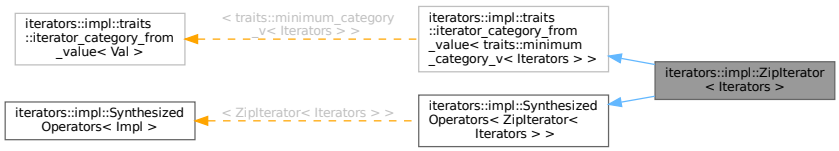
- Iterators.hpp

3.6 iterators::impl::ZipIterator< Iterators > Class Template Reference

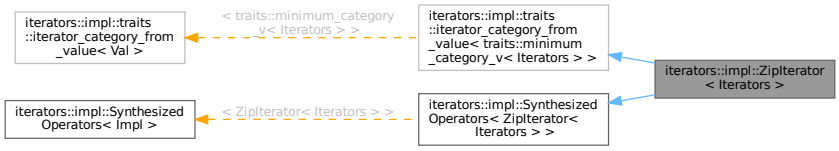
Class combining multiple iterators into one. Use it to iterate over multiple ranges at the same time.

`#include <Iterators.hpp>`

Inheritance diagram for iterators::impl::ZipIterator< Iterators >:



Collaboration diagram for iterators::impl::ZipIterator< Iterators >:



Public Types

- using **value_type** = traits::values_t< Iterators >
- using **reference** = traits::references_t< Iterators >
- using **pointer** = void
- using **difference_type** = std::ptrdiff_t

Public Member Functions

- constexpr **ZipIterator** (const Iterators &iterators) noexcept(std::is_nothrow_copy_constructible_v< Iterators >)
- template<typename Its = Iterators, typename = std::enable_if_t<traits::is_incrementable_v<Its>>>
  constexpr ZipIterator & operator++ () noexcept(traits::is_nothrow_incrementible_v< Iterators >)
- template<typename Its , REQUIRES(ZipIterator::oneEqual(INSTANCE_OF(Iterators), INSTANCE_OF(Its))) >
  constexpr bool operator== (const ZipIterator< Its > &other) const noexcept(noexcept(ZipIterator::one↩
  Equal(std::declval< Iterators >(), other.getIterators())))
- template<typename Its = Iterators, typename = std::enable_if_t<traits::is_dereferencible_v<Its>>>
  constexpr reference operator* () const noexcept(traits::is_nothrow_dereferencible_v< Iterators >)
- constexpr auto getIterators () const noexcept -> const Iterators &
- constexpr decltype(auto) operator[ ] (typename Implementation::difference_type n) const noexcept(noexcept(*(std↩
  ::declval< ZipIterator< Iterators > >()+n)))
- constexpr decltype(auto) operator[ ] (typename Implementation::difference_type n) noexcept(noexcept(*(std↩
  ::declval< ZipIterator< Iterators > >()+n)))
- constexpr ZipIterator< Iterators > operator++ (int) noexcept(noexcept(++std::declval< ZipIterator< Iterators > >()) &&std::is_nothrow_copy_constructible_v< ZipIterator< Iterators > >)

- constexpr ZipIterator< Iterators > operator-- (int) noexcept(noexcept(--std::declval< ZipIterator< Iterators > >()) &&std::is_nothrow_copy_constructible_v< ZipIterator< Iterators > >)
- constexpr bool operator!= (const T &other) const noexcept(noexcept(INSTANCE_OF_IMPL==other))
- constexpr bool operator<= (const T &rhs) const noexcept(noexcept(INSTANCE_OF_IMPL > rhs))
- constexpr bool operator>= (const T &rhs) const noexcept(noexcept(INSTANCE_OF_IMPL< rhs))

bidirectional iteration

the following operators are only available if all underlying iterators support bidirectional access

- template<bool IsBidirectional = traits::is_bidirectional_v<Iterators>>
  constexpr auto operator-- () noexcept(traits::is_nothrow_decrementible_v< Iterators >) -> std::enable_if_t< IsBidirectional, ZipIterator & >

random access operators

the following operators are only available if all underlying iterators support random access

- template<bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>>
  constexpr auto operator+= (difference_type n) noexcept(traits::is_nothrow_compound_assignable_plus_v< Iterators >) -> std::enable_if_t< IsRandomAccessible, ZipIterator & >
- template<bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>>
  constexpr auto operator-= (difference_type n) noexcept(traits::is_nothrow_compound_assignable_minus_v< Iterators >) -> std::enable_if_t< IsRandomAccessible, ZipIterator & >
- template<typename Its , bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>, REQUIRES(ZipIterator::minDifference(INSTANCE_OF(Iterators), INSTANCE_OF(Its))) >
  constexpr auto operator- (const ZipIterator< Its > &other) const -> std::enable_if_t< IsRandomAccessible, difference_type >
- template<typename Its , bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>, REQUIRES(ZipIterator::allLess(INSTANCE_OF(Iterators), INSTANCE_OF(Its))) >
  constexpr auto operator< (const ZipIterator< Its > &other) const noexcept(noexcept(ZipIterator::allLess(INSTANCE_OF(Iterators), INSTANCE_OF(Its)))) -> std::enable_if_t< IsRandomAccessible, bool >
- template<typename Its , bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>, REQUIRES(ZipIterator::allGreater(INSTANCE_OF(Iterators), INSTANCE_OF(Its))) >
  constexpr auto operator> (const ZipIterator< Its > &other) const noexcept(noexcept(ZipIterator::allGreater(INSTANCE_OF(Iterators), INSTANCE_OF(Its)))) -> std::enable_if_t< IsRandomAccessible, bool >

Related Symbols

(Note that these are not member symbols.)

- template<typename ... Iterators>
  constexpr auto zip_i (Iterators ...iterators) -> impl::ZipIterator< std::tuple< Iterators... > >

3.6.1 Detailed Description

template<typename Iterators>
class iterators::impl::ZipIterator< Iterators >

Class combining multiple iterators into one. Use it to iterate over multiple ranges at the same time.

ZipIterators only support the operators of the least powerful underling iterator. Zipping a random access iterator (e.g. from std::vector) and a bidirectional iterator (e.g. from std::list) results in a bidirectional iterator. All operators are SFINAE friendly.

ZipIterators return a tuple of references to the range elements. When using structured bindings, no additional reference binding is necessary.

Let `z` be a ZipIterator composed from two `std::vector<int>`

```
auto [val1, val2] = *z; // val1 and val2 are references to the vector elements
val1 = 17; // this will change the respective value in the first vector
```

Template Parameters

Iterators	Underlying iterator types

3.6.2 Member Function Documentation

getIterators()

```
template<typename Iterators >
constexpr auto iterators::impl::ZipIterator< Iterators >::getIterators ( ) const -> const
Iterators&   [inline], [constexpr], [noexcept]
```

Getter for underlying iterators

Returns

Const reference to underlying iterators

operator"!=()

```
constexpr bool iterators::impl::SynthesizedOperators< ZipIterator< Iterators > >::operator!=
(
            const T & other ) const  [inline], [constexpr], [noexcept], [inherited]
```

Inequality comparison

Template Parameters

T	type of right hand side
Implementation	SFINAE helper, do not specify explicitly

Parameters

other	right hand side

Returns

true if this is not equal to other

operator*()

```
template<typename Iterators >
template<typename Its = Iterators, typename = std::enable_if_t<traits::is_dereferencible_v<↩
Its>>>
constexpr reference iterators::impl::ZipIterator< Iterators >::operator* ( ) const  [inline],
[constexpr], [noexcept]
```

Dereferences all underlying iterators and returns a tuple of the resulting range reference types

Template Parameters

Its	SFINAE guard, do not specify

Returns

tuple of references to range elements

operator++() [1/2]

```
template<typename Iterators >
template<typename Its = Iterators, typename = std::enable_if_t<traits::is_incrementable_v<↩
Its>>>
constexpr ZipIterator & iterators::impl::ZipIterator< Iterators >::operator++ ( )  [inline],
[constexpr], [noexcept]
```

Increments all underlying iterators by one

Template Parameters

Its	SFINAE guard, do not specify

Returns

reference to this

operator++() [2/2]

```
constexpr ZipIterator< Iterators > iterators::impl::SynthesizedOperators< ZipIterator< Iterators
> >::operator++ (
            int  )  [inline], [constexpr], [noexcept], [inherited]
```

Postfix increment. Synthesized from prefix increment

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Returns

Instance of Impl

operator+=()

```
template<typename Iterators >
template<bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>>
```

```
constexpr auto iterators::impl::ZipIterator< Iterators >::operator+= (
            difference_type n ) -> std::enable_if_t<IsRandomAccessible, ZipIterator &>   [inline],
[constexpr], [noexcept]
```

Compound assignment increment. Increments all underlying iterators by n. Only available if all underlying iterators support at least random access

Template Parameters

IsRandomAccessible	SFINAE guard, do not specify

Parameters

n	increment

Returns

reference to this

operator-()

```
template<typename Iterators >
template<typename Its , bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>,
REQUIRES( ZipIterator::minDifference(INSTANCE_OF(Iterators), INSTANCE_OF(Its))) >
constexpr auto iterators::impl::ZipIterator< Iterators >::operator- (
            const ZipIterator< Its > & other ) const -> std::enable_if_t<IsRandomAccessible,
difference_type>   [inline], [constexpr]
```

Returns the minimum pairwise difference n between all underlying iterators of ∗this and other, such that (other + n) == ∗this Only available if all underlying iterators support at least random access

Template Parameters

Its	Iterator types of right hand side
IsRandomAccessible	SFINAE guard, do not specify

Parameters

other	right hand side

Returns

integer n such that (other + n) == ∗this

operator--() [1/2]

```
template<typename Iterators >
template<bool IsBidirectional = traits::is_bidirectional_v<Iterators>>
```

```
constexpr auto iterators::impl::ZipIterator< Iterators >::operator-- ( ) -> std::enable_if_↩
t<IsBidirectional, ZipIterator &>   [inline], [constexpr], [noexcept]
```

Decrements all underlying iterators by one. Only available if all iterators support at least bidirectional access

Template Parameters

IsBidirectional	SFINAE guard, do not specify

Returns

reference to this

operator--() [2/2]

```
constexpr ZipIterator< Iterators > iterators::impl::SynthesizedOperators< ZipIterator< Iterators
> >::operator-- (
            int  )  [inline], [constexpr], [noexcept], [inherited]
```

Postfix decrement. Synthesized from prefix decrement

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Returns

Instance of Impl

operator-=()

```
template<typename Iterators >
template<bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>>
constexpr auto iterators::impl::ZipIterator< Iterators >::operator-= (
            difference_type n ) -> std::enable_if_t<IsRandomAccessible, ZipIterator &>   [inline],
[constexpr], [noexcept]
```

Compound assignment decrement. Decrements all underlying iterators by n. Only available if all underlying iterators support at least random access

Template Parameters

IsRandomAccessible	SFINAE guard, do not specify

Parameters

n	decrement

Returns

reference to this

operator<()

```
template<typename Iterators >
template<typename Its , bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>,
REQUIRES( ZipIterator::allLess(INSTANCE_OF(Iterators), INSTANCE_OF(Its))) >
constexpr auto iterators::impl::ZipIterator< Iterators >::operator< (
            const ZipIterator< Its > & other ) const -> std::enable_if_t<IsRandomAccessible,
bool>   [inline], [constexpr], [noexcept]
```

Pairwise less comparison of underlying iterators Only available if all underlying iterators support at least random access

Template Parameters

Its	Iterator types of right hand side
IsRandomAccessible	SFINAE guard, do not specify

Parameters

other	right hand side

Returns

true if all underlying iterators compare less to the corresponding iterators from other

operator<=()

```
constexpr bool iterators::impl::SynthesizedOperators< ZipIterator< Iterators > >::operator<=
(
            const T & rhs ) const  [inline], [constexpr], [noexcept], [inherited]
```

Less than or equal comparison

Template Parameters

T	type of right hand side
Implementation	SFINAE helper, do not specify explicitly

Parameters

other	right hand side

Returns

true if this is not greater than other

operator==()

```
template<typename Iterators >
template<typename Its , REQUIRES(ZipIterator::oneEqual(INSTANCE_OF(Iterators), INSTANCE_↩
OF(Its))) >
constexpr bool iterators::impl::ZipIterator< Iterators >::operator== (
            const ZipIterator< Its > & other ) const  [inline], [constexpr], [noexcept]
```

Pairwise equality comparison of underlying iterators

Template Parameters

Its	Iterator types of right hand side

Parameters

other	right hand side

Returns

true if at least one underlying iterator compares equal to the corresponding iterator from other

operator>()

```
template<typename Iterators >
template<typename Its , bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>,
REQUIRES( ZipIterator::allGreater(INSTANCE_OF(Iterators), INSTANCE_OF(Its))) >
constexpr auto iterators::impl::ZipIterator< Iterators >::operator> (
            const ZipIterator< Its > & other ) const -> std::enable_if_t<IsRandomAccessible,
bool>   [inline], [constexpr], [noexcept]
```

Pairwise grater comparison of underlying iterators Only available if all underlying iterators support at least random access

Template Parameters

Its	Iterator types of right hand side
IsRandomAccessible	SFINAE guard, do not specify

Parameters

other	right hand side

Returns

true if all underlying iterators compare greater to the corresponding iterators from other

operator>=()

```
constexpr bool iterators::impl::SynthesizedOperators< ZipIterator< Iterators > >::operator>=
(
            const T & rhs ) const  [inline], [constexpr], [noexcept], [inherited]
```

Greater than or equal comparison

Template Parameters

T	type of right hand side
Implementation	SFINAE helper, do not specify explicitly

Parameters

other	right hand side

Returns

true if this is not less than other

operator[]() [1/2]

```
constexpr decltype(auto) iterators::impl::SynthesizedOperators< ZipIterator< Iterators > >↩
::operator[] (
            typename Implementation::difference_type n ) const  [inline], [constexpr], [noexcept],
[inherited]
```

Array subscript operator

Template Parameters

Implementation	SFINAE helper, do not specify explicitly

Parameters

n	index

Returns

∗(∗this + n)

operator[]() [2/2]

```
constexpr decltype(auto) iterators::impl::SynthesizedOperators< ZipIterator< Iterators > >↩
::operator[] (
            typename Implementation::difference_type n )  [inline], [constexpr], [noexcept],
[inherited]
```

3.6.3 Friends And Related Symbol Documentation

zip_i()

```
template<typename ...  Iterators>
constexpr auto zip_i (
            Iterators ...  iterators ) -> impl::ZipIterator<std::tuple<Iterators...>>   [related]
```

Function that is used to create a impl::ZipIterator from an arbitrary number of iterators

Template Parameters

Iterators	type of iterators

Parameters

iterators	arbitrary number of iterators

Returns

impl::ZipIterator

Note

ZipIterators have the same iterator category as the least powerful underlying operator. This means that for example, zipping a random access iterator and a bidirectional iterator only yields a bidirectional impl::ZipIterator

The documentation for this class was generated from the following file:

- Iterators.hpp

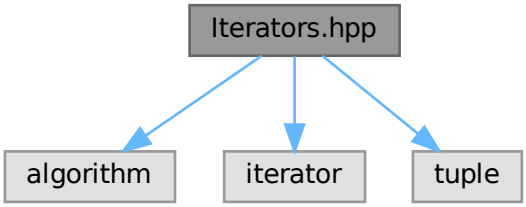
4 File Documentation

4.1 Iterators.hpp File Reference

This file contains the definitions of Python-like zip- and enumerate-functions. They can be used in range based for-loops to loop over multiple ranges at the same time, or to index a range while looping respectively.

```
#include <algorithm>
#include <iterator>
```

`#include <tuple>`
Include dependency graph for Iterators.hpp:



Classes

- struct iterators::impl::RefTuple< Ts >

  Proxy reference type that supports assignment and swap even on const instances.
- struct iterators::impl::SynthesizedOperators< Impl >

  CRTP-class that provides additional pointer arithmetic operators synthesized from basic operators.
- class iterators::impl::ZipIterator< Iterators >

  Class combining multiple iterators into one. Use it to iterate over multiple ranges at the same time.
- struct iterators::impl::Unreachable

  represents the unreachable end of an infinite sequence
- struct iterators::impl::CounterIterator< T >

  Iterator of an infinite sequence of numbers. Simply increments an internal counter.
- struct iterators::impl::CounterRange< T >

  Represents an infinite range of numbers.

Namespaces

- namespace iterators

  namespace containing zip and enumerate functions
- namespace iterators::impl

  namespace containing structures and helpers used to implement zip and enumerate. Normally there is no need to use any of its members directly
- namespace iterators::impl::traits

  namespace containing type traits used in implementation of zip and enumerate

Macros

- #define **REFERENCE**(TYPE) std::declval<std::add_lvalue_reference_t<TYPE>>()
- #define ALL_NOEXCEPT(OP, NAME)
- #define **ELEMENT1** std::get<Idx>(std::forward<Tuple1>(tuple1))
- #define **ELEMENT2** std::get<Idx>(std::forward<Tuple2>(tuple2))
- #define BINARY_TUPLE_FOR_EACH(OPERATION, NAME)
- #define **BINARY_TUPLE_FOR_EACH_FOLD**(OPERATION, COMBINATOR, NAME) BINARY_TUPLE_↩ FOR_EACH(((OPERATION) COMBINATOR ...), NAME)

- #define TYPE_MAP_DEFAULT
- #define TYPE_MAP(TYPE, VALUE)
- #define TYPE_MAP_ALIAS
- #define **INSTANCE_OF**(TYPENAME) std::declval<TYPENAME>()
- #define **INSTANCE_OF_IMPL** INSTANCE_OF(Implementation)
- #define **REQUIRES_IMPL**(TYPENAME, EXPRESSION) typename Implementation = TYPENAME, typename = std::void_t<decltype(EXPRESSION)>
- #define **REQUIRES**(EXPRESSION) typename = std::void_t<decltype(EXPRESSION)>
- #define **DERIVE_VIEW_INTERFACE**(CLASS)

Functions

- template<std::size_t Offset, std::size_t ... Idx>
  constexpr auto **iterators::impl::index_seq_impl** (std::index_sequence< Idx... >) noexcept
- template<std::size_t Start, std::size_t End>
  constexpr auto **iterators::impl::index_seq** () noexcept
- template<typename Tuple , std::size_t ... Idx1, std::size_t ... Idx2>
  constexpr auto **iterators::impl::tuple_split_impl** (Tuple &&tuple, std::index_sequence< Idx1... >, std::index_sequence< Idx2... >)
- template<std::size_t Idx, typename Tuple >
  constexpr auto **iterators::impl::tuple_split** (Tuple &&tuple)
- template<template< typename > typename Predicate, std::size_t Idx, typename ... Ts>
  constexpr std::size_t **iterators::impl::index_of** () noexcept
- template<typename Tuple >
  constexpr auto **iterators::impl::swap** (Tuple &&a, Tuple &&b) -> std::enable_if_t< iterators::impl::traits::is_same_template_v< iterators::impl::RefTuple, Tuple > >
- template<typename ... Iterable>
  struct ZipView iterators::impl::DERIVE_VIEW_INTERFACE (ZipView< Iterable... >)

  Zip-view that provides begin() and end() member functions. Use to loop over multiple ranges at the same time using ranged based for-loops.
- template<typename T >
  constexpr T iterators::impl::sgn (T val) noexcept
- template<typename ... Iterators>
  constexpr auto iterators::zip_i (Iterators ...iterators) -> impl::ZipIterator< std::tuple< Iterators... > >
- template<typename ... Iterable>
  constexpr auto iterators::zip (Iterable &&...iterable)
- template<typename ... Iterable>
  constexpr auto iterators::const_zip (Iterable &&...iterable)
- template<typename Container , typename T = std::size_t>
  constexpr auto iterators::enumerate (Container &&container, T start=T(0), T increment=T(1))
- template<typename Container , typename T = std::size_t>
  constexpr auto iterators::const_enumerate (Container &&container, T start=T(0), T increment=T(1))
- template<typename ... Iterable>
  constexpr auto iterators::zip_enumerate (Iterable &&...iterable)
- template<typename ... Iterable>
  constexpr auto iterators::const_zip_enumerate (Iterable &&...iterable)

4.1.1 Detailed Description

This file contains the definitions of Python-like zip- and enumerate-functions. They can be used in range based for-loops to loop over multiple ranges at the same time, or to index a range while looping respectively.

Author

tim Luchterhand

Date

10.09.21

4.1.2 Macro Definition Documentation

ALL_NOEXCEPT

```
#define ALL_NOEXCEPT(
               OP,
               NAME )
```

Value:

```
template<typename T> \
struct NAME { \
    static constexpr bool value = false; \
}; \
template<typename ...Ts> \
struct NAME <std::tuple<Ts...>> { \
    static constexpr bool value = (... && noexcept(OP)); \
};                  \
template<typename T> \
inline constexpr bool NAME##_v = NAME<T>::value;
```

BINARY_TUPLE_FOR_EACH

```
#define BINARY_TUPLE_FOR_EACH(
               OPERATION,
               NAME )
```

Value:

```
template<typename Tuple1, typename Tuple2, std::size_t ...Idx> \
static constexpr auto NAME##Impl(Tuple1 &&tuple1, Tuple2 &&tuple2, std::index_sequence<Idx...>) \
noexcept(noexcept((OPERATION))) -> decltype(OPERATION) { \
    return (OPERATION); \
} \
template<typename Tuple1, typename Tuple2> \
static constexpr auto NAME(Tuple1 &&tuple1, Tuple2 &&tuple2) \
noexcept(noexcept(NAME##Impl(std::forward<Tuple1>(tuple1), std::forward<Tuple2>(tuple2),        \
    std::make_index_sequence<std::tuple_size_v<std::remove_reference_t<Tuple1>>{}))) \
-> decltype(NAME##Impl(std::forward<Tuple1>(tuple1), std::forward<Tuple2>(tuple2),               \
    std::make_index_sequence<std::tuple_size_v<std::remove_reference_t<Tuple1>>{})) { \
    static_assert(std::tuple_size_v<std::remove_reference_t<Tuple1> ==
std::tuple_size_v<std::remove_reference_t<Tuple2>); \
    return NAME##Impl(std::forward<Tuple1>(tuple1), std::forward<Tuple2>(tuple2),                \
        std::make_index_sequence<std::tuple_size_v<std::remove_reference_t<Tuple1>>{}); \
}
```

TYPE_MAP

```
#define TYPE_MAP(
               TYPE,
               VALUE )
```

Value:

```
template<>            \
struct type_to_value<TYPE> { \
    static constexpr std::size_t value = VALUE; \
};                  \
template<>            \
struct value_to_type<VALUE>{ \
    static_assert(VALUE != 0, "0 is a reserved value"); \
    using type = TYPE;\
};
```

TYPE_MAP_ALIAS

```
#define TYPE_MAP_ALIAS
```

Value:

```
        template<typename T> \
        constexpr inline std::size_t type_to_value_v = type_to_value<T>::value; \
        template<std::size_t V>                                                  \
        using value_to_type_t = typename value_to_type<V>::type;
```

TYPE_MAP_DEFAULT

```
#define TYPE_MAP_DEFAULT
```

Value:

```
        template<typename> \
        struct type_to_value {}; \
        template<std::size_t>    \
        struct value_to_type {};
```

4.2 Iterators.hpp

Go to the documentation of this file.

```
00001 /**
00002  * @file Iterators.hpp
00003  * @author tim Luchterhand
00004  * @date 10.09.21
00005  * @brief This file contains the definitions of Python-like zip- and enumerate-functions. They can be
      used in range
00006  * based for-loops to loop over multiple ranges at the same time, or to index a range while looping
      respectively.
00007  */
00008
00009 #ifndef ITERATORTOOLS_ITERATORS_HPP
00010 #define ITERATORTOOLS_ITERATORS_HPP
00011
00012 #include <algorithm>
00013 #include <iterator>
00014 #include <tuple>
00015
00016 #define REFERENCE(TYPE) std::declval<std::add_lvalue_reference_t<TYPE»()
00017
00018 #define ALL_NOEXCEPT(OP, NAME)
00019         template<typename T>
00020         struct NAME {
00021             static constexpr bool value = false;
00022         };
00023         template<typename ...Ts>
00024         struct NAME <std::tuple<Ts...» {
00025             static constexpr bool value = (... && noexcept(OP));
00026         };
00027         template<typename T>
00028         inline constexpr bool NAME##_v = NAME<T>::value;
00029
00030 #define ELEMENT1 std::get<Idx>(std::forward<Tuple1>(tuple1))
00031 #define ELEMENT2 std::get<Idx>(std::forward<Tuple2>(tuple2))
00032
00033 #define BINARY_TUPLE_FOR_EACH(OPERATION, NAME)
00034         template<typename Tuple1, typename Tuple2, std::size_t ...Idx>
00035         static constexpr auto NAME##Impl(Tuple1 &&tuple1, Tuple2 &&tuple2,
      std::index_sequence<Idx...>)
00036         noexcept(noexcept((OPERATION))) -> decltype(OPERATION) {
00037             return (OPERATION);
00038         }
00039         template<typename Tuple1, typename Tuple2>
00040         static constexpr auto NAME(Tuple1 &&tuple1, Tuple2 &&tuple2)
00041         noexcept(noexcept(NAME##Impl(std::forward<Tuple1>(tuple1), std::forward<Tuple2>(tuple2),
00042             std::make_index_sequence<std::tuple_size_v<std::remove_reference_t<Tuple1»>{}))))
00043         -> decltype(NAME##Impl(std::forward<Tuple1>(tuple1), std::forward<Tuple2>(tuple2),
00044             std::make_index_sequence<std::tuple_size_v<std::remove_reference_t<Tuple1»>{})) {
00045             static_assert(std::tuple_size_v<std::remove_reference_t<Tuple1» ==
      std::tuple_size_v<std::remove_reference_t<Tuple2»);
00046             return NAME##Impl(std::forward<Tuple1>(tuple1), std::forward<Tuple2>(tuple2),
00047                 std::make_index_sequence<std::tuple_size_v<std::remove_reference_t<Tuple1»>{});
```

```
00048         }
00049
00050 #define BINARY_TUPLE_FOR_EACH_FOLD(OPERATION, COMBINATOR, NAME) BINARY_TUPLE_FOR_EACH( ( (OPERATION)
      COMBINATOR ...), NAME)
00051
00052 #define TYPE_MAP_DEFAULT
00053         template<typename>
00054         struct type_to_value {};
00055         template<std::size_t>
00056         struct value_to_type {};
00057
00058 #define TYPE_MAP(TYPE, VALUE)
00059         template<>
00060         struct type_to_value<TYPE> {
00061             static constexpr std::size_t value = VALUE;
00062         };
00063         template<>
00064         struct value_to_type<VALUE>{
00065             static_assert(VALUE != 0, "0 is a reserved value");
00066             using type = TYPE;
00067         };
00068
00069 #define TYPE_MAP_ALIAS
00070         template<typename T>
00071         constexpr inline std::size_t type_to_value_v = type_to_value<T>::value;
00072         template<std::size_t V>
00073         using value_to_type_t = typename value_to_type<V>::type;
00074
00075 #define INSTANCE_OF(TYPENAME) std::declval<TYPENAME>()
00076
00077 #define INSTANCE_OF_IMPL INSTANCE_OF(Implementation)
00078
00079 #define REQUIRES_IMPL(TYPENAME, EXPRESSION) typename Implementation = TYPENAME, typename =
      std::void_t<decltype(EXPRESSION)>
00080
00081 #define REQUIRES(EXPRESSION) typename = std::void_t<decltype(EXPRESSION)>
00082
00083 #ifndef __STD_RANGES_DISABLED__
00084 #ifdef __cpp_lib_ranges
00085 #include <ranges>
00086 #define DERIVE_VIEW_INTERFACE(CLASS) : std::ranges::view_interface<CLASS>
00087 #define __USE_VIEW_INTERFACE__
00088 #else
00089 #define DERIVE_VIEW_INTERFACE(CLASS)
00090 #endif
00091 #else
00092 #define DERIVE_VIEW_INTERFACE(CLASS)
00093 #endif
00094
00095 /**
00096  * @brief namespace containing zip and enumerate functions
00097  */
00098 namespace iterators {
00099
00100     /**
00101      * @brief namespace containing structures and helpers used to implement zip and enumerate.
00102      * Normally there is no need to use any of its members directly
00103      */
00104     namespace impl {
00105         /**
00106          * @brief Proxy reference type that supports assignment and swap even on const instances.
00107          * @details @copybrief
00108          * Actual constness is determined by element constness
00109          * @tparam Ts
00110          */
00111         template<typename ...Ts>
00112         struct RefTuple : public std::tuple<Ts...> {
00113             using std::tuple<Ts...>::tuple;
00114
00115             static constexpr bool Assignable = (not std::is_const_v<std::remove_reference_t<Ts» &&
      ...);
00116
00117             template<typename Dummy = int>
00118             constexpr auto operator=(RefTuple &&other)
      noexcept((std::is_nothrow_move_assignable_v<std::remove_reference_t<Ts» && ...))
00119                     -> std::enable_if_t<std::is_same_v<Dummy, int> and Assignable, RefTuple &> {
00120                 if (selfAssignGuard(other)) {
00121                     return *this;
00122                 }
00123
00124                 moveAssign(*this, other);
00125                 return *this;
00126             }
00127
00128             template<typename ...Us>
00129             decltype(auto) operator=(const std::tuple<Us...> &other) const {
00130                 if (selfAssignGuard(other)) {
```

```
00131                     return *this;
00132                 }
00133
00134                 copyAssign(*this, other);
00135                 return *this;
00136             }
00137
00138             template<typename ...Us>
00139             decltype(auto) operator=(std::tuple<Us...> &&other) const {
00140                 if (selfAssignGuard(other)) {
00141                     return *this;
00142                 }
00143
00144                 moveAssign(*this, other);
00145                 return *this;
00146             }
00147
00148             template<std::size_t Idx>
00149             constexpr decltype(auto) get() const & noexcept {
00150                 return std::get<Idx>(static_cast<std::tuple<Ts...>const &>(*this));
00151             }
00152
00153             template<std::size_t Idx>
00154             constexpr decltype(auto) get() & noexcept {
00155                 return std::get<Idx>(static_cast<std::tuple<Ts...> &>(*this));
00156             }
00157
00158             template<std::size_t Idx>
00159             constexpr decltype(auto) get() const && noexcept {
00160                 return std::get<Idx>(static_cast<std::tuple<Ts...> const &&>(*this));
00161             }
00162
00163             template<std::size_t Idx>
00164             constexpr decltype(auto) get() && noexcept {
00165                 return std::get<Idx>(static_cast<std::tuple<Ts...> &&>(*this));
00166             }
00167
00168             template<typename Tuple>
00169             constexpr auto swap(Tuple &&other) const -> std::enable_if_t<std::is_same_v<Tuple,
    RefTuple> and Assignable> {
00170                 using NonConstTuple = std::remove_const_t<std::remove_reference_t<Tuple»;
00171                 auto * const _this = const_cast<RefTuple *>(this);
00172                 static_cast<std::tuple<Ts...>*>(_this)->swap(const_cast<NonConstTuple&>(other));
00173             }
00174
00175
00176         private:
00177             template<typename T>
00178             constexpr bool selfAssignGuard(const T &t) const noexcept {
00179                 if constexpr (std::is_same_v<T, RefTuple>) {
00180                     return &t == this;
00181                 } else {
00182                     return false;
00183                 }
00184             }
00185
00186             BINARY_TUPLE_FOR_EACH(((ELEMENT1 = ELEMENT2), ...), copyAssign)
00187             BINARY_TUPLE_FOR_EACH(((ELEMENT1 = std::move(ELEMENT2)), ...), moveAssign)
00188         };
00189
00190         /**
00191          * @brief namespace containing type traits used in implementation of zip and enumerate
00192          */
00193         namespace traits {
00194             template<bool Cond, typename T>
00195             using reference_if_t = std::conditional_t<Cond, std::add_lvalue_reference_t<T>, T>;
00196
00197             template<bool Cond, typename T>
00198             using const_if_t = std::conditional_t<Cond, std::add_const_t<T>, T>;
00199
00200             template<typename T, typename = std::void_t<»
00201             struct is_container : std::false_type {};
00202
00203             template<typename T>
00204             struct is_container<T, std::void_t<decltype(std::begin(std::declval<T>()),
    std::end(std::declval<T>()))»
00205                     : std::true_type {};
00206
00207             template<typename T>
00208             constexpr inline bool is_container_v = is_container<T>::value;
00209
00210             template<typename T, typename = std::void_t<»
00211             struct is_dereferencible : std::false_type {};
00212
00213             template<typename T>
00214             struct is_dereferencible<T, std::void_t<decltype(*std::declval<T>())» : std::true_type {};
00215
```

```
00216             template<typename ...Ts>
00217             struct is_dereferencible<std::tuple<Ts...>, void> {
00218                 static constexpr bool value = (is_dereferencible<Ts>::value && ...);
00219             };
00220 
00221             template<typename T>
00222             constexpr inline bool is_dereferencible_v = is_dereferencible<T>::value;
00223 
00224             template<typename T, typename = std::void_t<»
00225             struct is_incrementable : std::false_type {};
00226 
00227             template<typename T>
00228             struct is_incrementable<T, std::void_t<decltype(++REFERENCE(T))» : std::true_type {};
00229 
00230             template<typename ...Ts>
00231             struct is_incrementable<std::tuple<Ts...>, void> {
00232                 static constexpr bool value = (is_incrementable<Ts>::value && ...);
00233             };
00234 
00235             template<typename T>
00236             constexpr inline bool is_incrementable_v = is_incrementable<T>::value;
00237 
00238             template<typename T, bool B>
00239             struct dereference {
00240                 using type = void;
00241             };
00242 
00243             template<typename T>
00244             struct dereference<T, true> {
00245                 using type = decltype(*std::declval<T>());
00246             };
00247 
00248             template<typename T>
00249             using dereference_t = typename dereference<T, is_dereferencible_v<T»::type;
00250 
00251             template<typename T>
00252             struct values{};
00253 
00254             template<typename ...Ts>
00255             struct values<std::tuple<Ts...» {
00256                 using type =
      std::tuple<std::remove_const_t<std::remove_reference_t<dereference_t<Ts»»...>;
00257             };
00258 
00259             template<typename T>
00260             using values_t = typename values<T>::type;
00261 
00262             template<typename T>
00263             struct references{};
00264 
00265             template<typename ...Ts>
00266             struct references<std::tuple<Ts...» {
00267                 using type = RefTuple<dereference_t<Ts>...>;
00268             };
00269 
00270             template<typename T>
00271             using references_t = typename references<T>::type;
00272 
00273             ALL_NOEXCEPT(++REFERENCE(Ts), is_nothrow_incrementible)
00274             ALL_NOEXCEPT(--REFERENCE(Ts), is_nothrow_decrementible)
00275             ALL_NOEXCEPT(*REFERENCE(Ts), is_nothrow_dereferencible)
00276             ALL_NOEXCEPT(REFERENCE(Ts) += 5, is_nothrow_compound_assignable_plus)
00277             ALL_NOEXCEPT(REFERENCE(Ts) -= 5, is_nothrow_compound_assignable_minus)
00278 
00279             TYPE_MAP_DEFAULT
00280 
00281             TYPE_MAP(std::input_iterator_tag, 1)
00282             TYPE_MAP(std::forward_iterator_tag, 2)
00283             TYPE_MAP(std::bidirectional_iterator_tag, 3)
00284             TYPE_MAP(std::random_access_iterator_tag, 4)
00285             #if __cplusplus > 201703L
00286             TYPE_MAP(std::contiguous_iterator_tag, 5)
00287             #endif
00288 
00289             TYPE_MAP_ALIAS
00290 
00291             template<typename T, typename = std::void_t<»
00292             struct iterator_category_value {
00293                 static constexpr std::size_t value = 0;
00294             };
00295 
00296             template<typename T>
00297             struct iterator_category_value<T, std::void_t<typename
      std::iterator_traits<T>::iterator_category» {
00298                 static constexpr std::size_t value = type_to_value_v<typename
      std::iterator_traits<T>::iterator_category>;
00299             };
```

```
00300
00301             template<std::size_t Val>
00302             struct iterator_category_from_value {
00303                 using iterator_category = value_to_type_t<Val>;
00304             };
00305
00306             template<>
00307             struct iterator_category_from_value<0> {};
00308
00309             template<typename T>
00310             struct minimum_category {};
00311
00312             template<typename ...Ts>
00313             struct minimum_category<std::tuple<Ts...>> {
00314                 static constexpr std::size_t value =
      std::min({iterator_category_value<Ts>::value...});
00315             };
00316
00317             template<typename T>
00318             constexpr inline std::size_t minimum_category_v = minimum_category<T>::value;
00319
00320             template<typename T, typename = std::void_t<>>
00321             struct is_random_accessible {
00322                 static constexpr bool value = false;
00323             };
00324
00325             template<typename T>
00326             struct is_random_accessible<T, std::void_t<typename
      std::iterator_traits<T>::iterator_category>> {
00327                 static constexpr bool value = std::is_base_of_v<std::random_access_iterator_tag,
00328                         typename std::iterator_traits<T>::iterator_category>;
00329             };
00330
00331             template<typename ...Ts>
00332             struct is_random_accessible<std::tuple<Ts...>,
      std::void_t<value_to_type_t<minimum_category_v<std::tuple<Ts...>>>> {
00333                 static constexpr bool value = std::is_base_of_v<std::random_access_iterator_tag,
00334                         value_to_type_t<minimum_category_v<std::tuple<Ts...>>>;
00335             };
00336
00337             template<typename T>
00338             constexpr inline bool is_random_accessible_v = is_random_accessible<T>::value;
00339
00340             template<typename T, typename = std::void_t<>>
00341             struct is_bidirectional {
00342                 static constexpr bool value = false;
00343             };
00344
00345             template<typename T>
00346             struct is_bidirectional<T, std::void_t<typename
      std::iterator_traits<T>::iterator_category>> {
00347                 static constexpr bool value = std::is_base_of_v<std::bidirectional_iterator_tag,
00348                         typename std::iterator_traits<T>::iterator_category>;
00349             };
00350
00351             template<typename ...Ts>
00352             struct is_bidirectional<std::tuple<Ts...>,
      std::void_t<value_to_type_t<minimum_category_v<std::tuple<Ts...>>>> {
00353                 static constexpr bool value = std::is_base_of_v<std::bidirectional_iterator_tag,
00354                         value_to_type_t<minimum_category_v<std::tuple<Ts...>>>;
00355             };
00356
00357             template<typename T>
00358             constexpr inline bool is_bidirectional_v = is_bidirectional<T>::value;
00359
00360             template<typename T, typename = std::void_t<>>
00361             struct has_size : std::false_type {};
00362
00363             template<typename T>
00364             struct has_size<T,
      std::void_t<decltype(std::size(std::declval<std::remove_reference_t<T>>()))>
00365                     : std::true_type {};
00366
00367             template<typename ...Ts>
00368             struct has_size<std::tuple<Ts...>> {
00369                 static constexpr bool value = (has_size<Ts>::value &&...);
00370             };
00371
00372             template<typename T>
00373             constexpr inline bool has_size_v = has_size<T>::value;
00374
00375             template<template<typename...> typename Template, typename T>
00376             struct is_same_template : std::false_type {};
00377
00378             template<template<typename...> typename Template, typename... Args>
00379             struct is_same_template<Template, Template<Args...>> : std::true_type {};
00380
```

```
00381             template<template<typename ...> typename Template, typename T>
00382             constexpr inline bool is_same_template_v = is_same_template<Template,
00383                     std::remove_const_t<std::remove_reference_t<T»>::value;
00384         }
00385
00386
00387         template<std::size_t Offset, std::size_t ...Idx>
00388         constexpr auto index_seq_impl(std::index_sequence<Idx...>) noexcept {
00389             return std::index_sequence<(Idx + Offset)...>{};
00390         }
00391
00392         template<std::size_t Start, std::size_t End>
00393         constexpr auto index_seq() noexcept {
00394             return index_seq_impl<Start>(std::make_index_sequence<End - Start>());
00395         }
00396
00397         template<typename Tuple, std::size_t ...Idx1, std::size_t ...Idx2>
00398         constexpr auto tuple_split_impl(Tuple &&tuple, std::index_sequence<Idx1...>,
      std::index_sequence<Idx2...>) {
00399             return std::make_pair(std::tuple<std::tuple_element_t<Idx1,
      Tuple>&&...>(std::get<Idx1>(std::forward<Tuple>(tuple))...),
00400                                   std::tuple<std::tuple_element_t<Idx2,
      Tuple>&&...>(std::get<Idx2>(std::forward<Tuple>(tuple))...));
00401         }
00402
00403         template<std::size_t Idx, typename Tuple>
00404         constexpr auto tuple_split(Tuple &&tuple) {
00405             return tuple_split_impl(std::forward<Tuple>(tuple),
00406                                     std::make_index_sequence<Idx>(),
00407                                     index_seq<Idx, std::tuple_size_v<Tuple»());
00408         }
00409
00410         template<template<typename> typename Predicate, std::size_t Idx, typename ...Ts>
00411         constexpr std::size_t index_of() noexcept {
00412             static_assert(Idx < sizeof...(Ts));
00413             if constexpr (Predicate<std::tuple_element_t<Idx, std::tuple<Ts...»>::value) {
00414                 return Idx;
00415             } else {
00416                 return index_of<Predicate, Idx + 1, Ts...>();
00417             }
00418         }
00419
00420         template<typename Tuple>
00421         constexpr auto swap(Tuple &&a, Tuple &&b)
00422         -> std::enable_if_t<iterators::impl::traits::is_same_template_v<iterators::impl::RefTuple,
      Tuple» {
00423             std::forward<Tuple>(a).swap(std::forward<Tuple>(b));
00424         }
00425
00426
00427         /**
00428          * @brief CRTP-class that provides additional pointer arithmetic operators synthesized from
      basic operators
00429          * @details @copybrief
00430          * Adds the following operators
00431          * - postfix increment and decrement (requires the respective prefix operators)
00432          * - array subscript operator[] (requires operator+ and dereference operator)
00433          * - binary arithmetic operators (requires compound assignment operators)
00434          * - inequality comparison (requires operator==)
00435          * - less than or equal comparison (requires operator>)
00436          * - greater than or equal comparison (requires operator<)
00437          * @tparam Impl Base class
00438          */
00439         template<typename Impl>
00440         struct SynthesizedOperators {
00441
00442             /**
00443              * Array subscript operator
00444              * @tparam Implementation SFINAE helper, do not specify explicitly
00445              * @param n index
00446              * @return *(*this + n)
00447              */
00448             template<REQUIRES_IMPL(Impl, *(INSTANCE_OF_IMPL + INSTANCE_OF(typename
      Implementation::difference_type)))>
00449             constexpr decltype(auto) operator[](typename Implementation::difference_type n) const
00450             noexcept(noexcept(*(std::declval<Impl>() + n))) {
00451                 return *(this->getImpl() + n);
00452             }
00453
00454             /**
00455              * @copydoc SynthesizedOperators::operator[](typename Implementation::difference_type n)
00456              */
00457             template<REQUIRES_IMPL(Impl, *(INSTANCE_OF_IMPL + INSTANCE_OF(typename
      Implementation::difference_type)))>
00458             constexpr decltype(auto) operator[](typename Implementation::difference_type n)
00459             noexcept(noexcept(*(std::declval<Impl>() + n))) {
00460                 return *(this->getImpl() + n);
```

```
00461             }
00462
00463             /**
00464              * Postfix increment. Synthesized from prefix increment
00465              * @tparam Implementation SFINAE helper, do not specify explicitly
00466              * @return Instance of Impl
00467              */
00468             template<REQUIRES_IMPL(Impl, ++INSTANCE_OF_IMPL)>
00469             constexpr Impl operator++(int)
00470             noexcept(noexcept(++std::declval<Impl>()) && std::is_nothrow_copy_constructible_v<Impl>) {
00471                 auto tmp = this->getImpl();
00472                 this->getImpl().operator++();
00473                 return tmp;
00474             }
00475
00476             /**
00477              * Postfix decrement. Synthesized from prefix decrement
00478              * @tparam Implementation SFINAE helper, do not specify explicitly
00479              * @return Instance of Impl
00480              */
00481             template<REQUIRES_IMPL(Impl, --INSTANCE_OF_IMPL)>
00482             constexpr Impl operator--(int)
00483             noexcept(noexcept(--std::declval<Impl>()) && std::is_nothrow_copy_constructible_v<Impl>) {
00484                 auto tmp = this->getImpl();
00485                 this->getImpl().operator--();
00486                 return tmp;
00487             }
00488
00489             /**
00490              * Binary +plus operator. Synthesized from compound assignment operator+=
00491              * @tparam Implementation SFINAE helper, do not specify explicitly
00492              * @param it left hand side
00493              * @param n right hand side
00494              * @return Instance of Impl
00495              */
00496             template<REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL += INSTANCE_OF(typename
      Implementation::difference_type)) >
00497             friend constexpr auto operator+(Impl it, typename Implementation::difference_type n)
00498             noexcept(noexcept(std::declval<Impl>() += n)) {
00499                 it += n;
00500                 return it;
00501             }
00502
00503             /**
00504              * Binary +plus operator. Synthesized from compound assignment operator+=
00505              * @tparam Implementation SFINAE helper, do not specify explicitly
00506              * @param n left hand side
00507              * @param it right hand side
00508              * @return Instance of Impl
00509              */
00510             template<REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL += INSTANCE_OF(typename
      Implementation::difference_type))>
00511             friend constexpr auto operator+(typename Implementation::difference_type n, Impl it)
00512             noexcept(noexcept(std::declval<Impl>() += n)) {
00513                 it += n;
00514                 return it;
00515             }
00516
00517             /**
00518              * Binary minus operator. Synthesized from compound assignment operator-=
00519              * @tparam Implementation SFINAE helper, do not specify explicitly
00520              * @param it left hand side
00521              * @param n right hand side
00522              * @return Instance of Impl
00523              */
00524             template<REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL -= INSTANCE_OF(typename
      Implementation::difference_type)) >
00525             friend constexpr auto operator-(Impl it, typename Implementation::difference_type n)
00526             noexcept(noexcept(std::declval<Impl>() -= n)) {
00527                 it -= n;
00528                 return it;
00529             }
00530
00531             /**
00532              * Inequality comparison
00533              * @tparam T type of right hand side
00534              * @tparam Implementation SFINAE helper, do not specify explicitly
00535              * @param other right hand side
00536              * @return true if this is not equal to other
00537              */
00538             template<typename T, REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL == INSTANCE_OF(T))>
00539             constexpr bool operator!=(const T &other) const noexcept(noexcept(INSTANCE_OF_IMPL ==
      other)) {
00540                 return !(this->getImpl()== other);
00541             }
00542
00543             /**
```

```
00544              * Less than or equal comparison
00545              * @tparam T type of right hand side
00546              * @tparam Implementation SFINAE helper, do not specify explicitly
00547              * @param other right hand side
00548              * @return true if this is not greater than other
00549              */
00550             template<typename T, REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL > INSTANCE_OF(T))>
00551             constexpr bool operator<=(const T& rhs) const noexcept(noexcept(INSTANCE_OF_IMPL > rhs)) {
00552                 return !(this->getImpl()> rhs);
00553             }
00554
00555             /**
00556              * Greater than or equal comparison
00557              * @tparam T type of right hand side
00558              * @tparam Implementation SFINAE helper, do not specify explicitly
00559              * @param other right hand side
00560              * @return true if this is not less than other
00561              */
00562             template<typename T, REQUIRES_IMPL(Impl, INSTANCE_OF_IMPL < INSTANCE_OF(T))>
00563             constexpr bool operator>=(const T& rhs) const noexcept(noexcept(INSTANCE_OF_IMPL < rhs)) {
00564                 return !(this->getImpl() < rhs);
00565             }
00566
00567         private:
00568             constexpr Impl &getImpl() noexcept {
00569                 return static_cast<Impl &>(*this);
00570             }
00571
00572             constexpr const Impl &getImpl() const noexcept {
00573                 return static_cast<const Impl &>(*this);
00574             }
00575
00576             SynthesizedOperators() = default;
00577             friend Impl;
00578         };
00579
00580         /**
00581          * @brief Class combining multiple iterators into one. Use it to iterate over multiple ranges
     at the same time.
00582          * @details @copybrief
00583          * ZipIterators only support the operators of the least powerful underling iterator. Zipping a
     random access
00584          * iterator (e.g. from std::vector) and a bidirectional iterator (e.g. from std::list) results
     in a
00585          * bidirectional iterator. All operators are SFINAE friendly.
00586          *
00587          * ZipIterators return a tuple of references to the range elements. When using
00588          * structured bindings, no additional reference binding is necessary.
00589          *
00590          * Let ```z"` be a ZipIterator composed from two ```std::vector<int>"`
00591          * ```
00592          * auto [val1, val2] = *z; // val1 and val2 are references to the vector elements
00593          * val1 = 17; // this will change the respective value in the first vector
00594          * ```
00595          * @tparam Iterators Underlying iterator types
00596          */
00597         template<typename Iterators>
00598         class ZipIterator
00599                 : public traits::iterator_category_from_value<traits::minimum_category_v<Iterators»,
00600                   public SynthesizedOperators<ZipIterator<Iterators» {
00601
00602         public:
00603             using value_type = traits::values_t<Iterators>;
00604             using reference = traits::references_t<Iterators>;
00605             using pointer = void;
00606             using difference_type = std::ptrdiff_t;
00607
00608         private:
00609             BINARY_TUPLE_FOR_EACH_FOLD(ELEMENT1 == ELEMENT2, ||, oneEqual)
00610             BINARY_TUPLE_FOR_EACH_FOLD(ELEMENT1 < ELEMENT2, &&, allLess)
00611             BINARY_TUPLE_FOR_EACH_FOLD(ELEMENT1 > ELEMENT2, &&, allGreater)
00612             BINARY_TUPLE_FOR_EACH(std::min<difference_type>({ELEMENT1 - ELEMENT2 ...}), minDifference)
00613             Iterators iterators;
00614
00615         public:
00616             using SynthesizedOperators<ZipIterator>::operator++;
00617             using SynthesizedOperators<ZipIterator>::operator--;
00618
00619             constexpr ZipIterator() noexcept = default;
00620
00621             explicit constexpr ZipIterator(
00622                     const Iterators &iterators)
     noexcept(std::is_nothrow_copy_constructible_v<Iterators>)
00623                     : iterators(iterators) {}
00624
00625             /**
00626              * Increments all underlying iterators by one
```

```
00627              * @tparam Its SFINAE guard, do not specify
00628              * @return reference to this
00629              */
00630             template<typename Its = Iterators, typename =
      std::enable_if_t<traits::is_incrementable_v<Its>>
00631             constexpr ZipIterator &operator++()
      noexcept(traits::is_nothrow_incrementible_v<Iterators>) {
00632                 std::apply([](auto &&...it) { (++it, ...); }, iterators);
00633                 return *this;
00634             }
00635
00636             /**
00637              * @name bidirectional iteration
00638              * @brief the following operators are only available if all underlying iterators support
      bidirectional
00639              * access
00640              */
00641             ///@{
00642
00643             /**
00644              * Decrements all underlying iterators by one. Only available if all iterators support at
      least
00645              * bidirectional access
00646              * @tparam IsBidirectional SFINAE guard, do not specify
00647              * @return reference to this
00648              */
00649             template<bool IsBidirectional = traits::is_bidirectional_v<Iterators>>
00650             constexpr auto operator--() noexcept(traits::is_nothrow_decrementible_v<Iterators>)
00651                 -> std::enable_if_t<IsBidirectional, ZipIterator &> {
00652                 std::apply([](auto &&...it) { (--it, ...); }, iterators);
00653                 return *this;
00654             }
00655
00656             ///@}
00657
00658             /**
00659              * @name random access operators
00660              * @brief the following operators are only available if all underlying iterators support
      random access
00661              *
00662              */
00663             ///@{
00664
00665             /**
00666              * Compound assignment increment. Increments all underlying iterators by n. Only available
      if all underlying
00667              * iterators support at least random access
00668              * @tparam IsRandomAccessible SFINAE guard, do not specify
00669              * @param n increment
00670              * @return reference to this
00671              */
00672             template<bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>>
00673             constexpr auto operator+=(difference_type n)
      noexcept(traits::is_nothrow_compound_assignable_plus_v<Iterators>)
00674                     -> std::enable_if_t<IsRandomAccessible, ZipIterator &> {
00675                 std::apply([n](auto &&...it) {((it += n), ...);}, iterators);
00676                 return *this;
00677             }
00678
00679             /**
00680              * Compound assignment decrement. Decrements all underlying iterators by n. Only available
      if all underlying
00681              * iterators support at least random access
00682              * @tparam IsRandomAccessible SFINAE guard, do not specify
00683              * @param n decrement
00684              * @return reference to this
00685              */
00686             template<bool IsRandomAccessible = traits::is_random_accessible_v<Iterators>>
00687             constexpr auto operator-=(difference_type n)
      noexcept(traits::is_nothrow_compound_assignable_minus_v<Iterators>)
00688                     -> std::enable_if_t<IsRandomAccessible, ZipIterator &> {
00689                 std::apply([n](auto &&...it) {((it -= n), ...);}, iterators);
00690                 return *this;
00691             }
00692
00693             /**
00694              * Returns the minimum pairwise difference n between all underlying iterators of *this and
      other, such that
00695              * (other + n) == *this
00696              * Only available if all underlying iterators support at least random access
00697              * @tparam Its Iterator types of right hand side
00698              * @tparam IsRandomAccessible SFINAE guard, do not specify
00699              * @param other right hand side
00700              * @return integer n such that (other + n) == *this
00701              */
00702             template<typename Its, bool IsRandomAccessible =
      traits::is_random_accessible_v<Iterators>, REQUIRES(
```

```
00703                   ZipIterator::minDifference(INSTANCE_OF(Iterators), INSTANCE_OF(Its)))>
00704             constexpr auto operator-(const ZipIterator<Its> &other) const
00705             -> std::enable_if_t<IsRandomAccessible, difference_type> {
00706                 return minDifference(iterators, other.getIterators());
00707             }
00708
00709             /**
00710              * Pairwise less comparison of underlying iterators
00711              * Only available if all underlying iterators support at least random access
00712              * @tparam Its Iterator types of right hand side
00713              * @tparam IsRandomAccessible SFINAE guard, do not specify
00714              * @param other right hand side
00715              * @return true if all underlying iterators compare less to the corresponding iterators
    from other
00716              */
00717             template<typename Its, bool IsRandomAccessible =
    traits::is_random_accessible_v<Iterators>, REQUIRES(
00718                   ZipIterator::allLess(INSTANCE_OF(Iterators), INSTANCE_OF(Its)))>
00719             constexpr auto operator<(const ZipIterator<Its> &other) const
00720             noexcept(noexcept(ZipIterator::allLess(INSTANCE_OF(Iterators), INSTANCE_OF(Its))))
00721             -> std::enable_if_t<IsRandomAccessible, bool> {
00722                 return allLess(iterators, other.getIterators());
00723             }
00724
00725             /**
00726              * Pairwise grater comparison of underlying iterators
00727              * Only available if all underlying iterators support at least random access
00728              * @tparam Its Iterator types of right hand side
00729              * @tparam IsRandomAccessible SFINAE guard, do not specify
00730              * @param other right hand side
00731              * @return true if all underlying iterators compare greater to the corresponding iterators
    from other
00732              */
00733             template<typename Its, bool IsRandomAccessible =
    traits::is_random_accessible_v<Iterators>, REQUIRES(
00734                   ZipIterator::allGreater(INSTANCE_OF(Iterators), INSTANCE_OF(Its))) >
00735             constexpr auto operator>(const ZipIterator<Its> &other) const
    noexcept(noexcept(ZipIterator::allGreater(
00736                   INSTANCE_OF(Iterators), INSTANCE_OF(Its)))) ->
    std::enable_if_t<IsRandomAccessible, bool> {
00737                 return allGreater(iterators, other.getIterators());
00738             }
00739
00740             ///@}
00741
00742             /**
00743              * Pairwise equality comparison of underlying iterators
00744              * @tparam Its Iterator types of right hand side
00745              * @param other right hand side
00746              * @return true if at least one underlying iterator compares equal to the corresponding
    iterator from other
00747              */
00748             template<typename Its, REQUIRES(ZipIterator::oneEqual(INSTANCE_OF(Iterators),
    INSTANCE_OF(Its)))>
00749             constexpr bool operator==(const ZipIterator<Its> &other) const
00750             noexcept(noexcept(ZipIterator::oneEqual(std::declval<Iterators>(), other.getIterators())))
    {
00751                 return oneEqual(iterators, other.getIterators());
00752             }
00753
00754             /**
00755              * Dereferences all underlying iterators and returns a tuple of the resulting range
    reference types
00756              * @tparam Its SFINAE guard, do not specify
00757              * @return tuple of references to range elements
00758              */
00759             template<typename Its = Iterators, typename =
    std::enable_if_t<traits::is_dereferencible_v<Its>>>
00760             constexpr reference operator*() const
    noexcept(traits::is_nothrow_dereferencible_v<Iterators>) {
00761                 return std::apply([](auto &&...it) { return reference(*it...); }, iterators);
00762             }
00763
00764             /**
00765              * Getter for underlying iterators
00766              * @return Const reference to underlying iterators
00767              */
00768             constexpr auto getIterators() const noexcept -> const Iterators& {
00769                 return iterators;
00770             }
00771         };
00772
00773         /**
00774          * @brief Zip-view that provides begin() and end() member functions. Use to loop over multiple
    ranges at the
00775          * same time using ranged based for-loops.
00776          * @details @copybrief
```

```
00777          * Ranges are captured by lvalue reference, no copying occurs. Temporaries are allowed as well
       in which case
00778          * storage is moved into the zip-view.
00779          * @tparam Iterable Underlying range types
00780          */
00781         template<typename ...Iterable>
00782         struct ZipView DERIVE_VIEW_INTERFACE(ZipView<Iterable...>) {
00783         private:
00784             using ContainerTuple = std::tuple<Iterable...>;
00785             template<bool Const>
00786             using Iterators = std::tuple<decltype(std::begin(
00787                     std::declval<std::add_lvalue_reference_t<traits::const_if_t<Const,
       std::remove_reference_t<Iterable>>>()))...>;
00788             template<bool Const>
00789             using Sentinels = std::tuple<decltype(std::end(
00790                     std::declval<std::add_lvalue_reference_t<traits::const_if_t<Const,
       std::remove_reference_t<Iterable>>>()))...>;
00791             using IteratorTuple = Iterators<false>;
00792             using SentinelTuple = Sentinels<false>;
00793 
00794             template<typename Tuple, std::size_t ...Idx>
00795             constexpr auto sizeImpl(const Tuple &contTuple, std::index_sequence<Idx...>) const {
00796                 return std::min({std::size(std::get<Idx>(contTuple))...});
00797             }
00798         public:
00799             /**
00800              * CTor. Binds reference to ranges or takes ownership in case of rvalue references
00801              * @tparam Container range types
00802              * @param containers arbitrary number of ranges
00803              */
00804             template<typename ...Container>
00805             constexpr explicit ZipView(Container &&...containers) :
00806                 containers(std::forward<Container>(containers)...) {}
00807 
00808             ZipView() = default;
00809 
00810 
00811             /**
00812             * Returns a ZipIterator to the first elements of the underlying ranges
00813             * @return ZipIterator created by invoking std::begin on all underlying ranges
00814             */
00815             constexpr auto begin() {
00816                 return ZipIterator<IteratorTuple>(
00817                         std::apply([](auto &&...c) { return IteratorTuple(std::begin(c)...); },
       containers));
00818             }
00819 
00820             /**
00821             * Returns a ZipIterator to the elements following the last elements of the the underlying
       ranges
00822             * @return ZipIterator created by invoking std::end on all underlying ranges
00823             */
00824             constexpr auto end() {
00825                 return ZipIterator<SentinelTuple>(
00826                         std::apply([](auto &&...c) { return SentinelTuple(std::end(c)...); },
       containers));
00827             }
00828 
00829             /**
00830              * @copydoc ZipView::begin()
00831              * @note returns a ZipIterator that does not allow changing the ranges' elements
00832              */
00833             template<bool C = true>
00834             constexpr auto begin() const -> ZipIterator<Iterators<C>> {
00835                 return ZipIterator<Iterators<true>>(
00836                         std::apply([](auto &&...c) { return Iterators<true>(std::begin(c)...); },
       containers));
00837             }
00838 
00839             /**
00840              * @copydoc ZipView::end()
00841              */
00842             template<bool C = true>
00843             constexpr auto end() const -> ZipIterator<Sentinels<C>> {
00844                 return ZipIterator<Sentinels<true>>(
00845                         std::apply([](auto &&...c) { return Sentinels<true>(std::end(c)...); },
       containers));
00846             }
00847 
00848 #ifndef __USE_VIEW_INTERFACE__
00849             /**
00850              * Array subscript operator (no bounds are checked)
00851              * @tparam IsRandomAccess SFINAE helper, do not specify explicitly
00852              * @param index index
00853              * @return zip view element at given index
00854              */
00855             template<bool IsRandomAccess = traits::is_random_accessible_v<IteratorTuple>,
```

```
00856                 typename = std::enable_if_t<IsRandomAccess»
00857             constexpr auto operator[](std::size_t index) {
00858                 return begin()[index];
00859             }
00860 
00861             /**
00862              * @copydoc ZipView::operator[](std::size_t index)
00863              */
00864             template<bool C = true, bool IsRandomAccess = traits::is_random_accessible_v<Iterators<C»,
00865                 typename = std::enable_if_t<IsRandomAccess»
00866             constexpr auto operator[](std::size_t index) const {
00867                 return begin()[index];
00868             }
00869 
00870 
00871             /**
00872              * Returns the smallest size of all containers. Only available if all containers know
      their size
00873              * @tparam HasSize SFINAE guard, do not specify explicitly
00874              * @return smallest size of all containers
00875              */
00876             template<bool HasSize = traits::has_size_v<ContainerTuple»
00877             constexpr auto size() const -> std::enable_if_t<HasSize, std::size_t> {
00878                 return sizeImpl(containers,
      std::make_index_sequence<std::tuple_size_v<ContainerTuple»());
00879             }
00880 
00881 #endif
00882 
00883         private:
00884             ContainerTuple containers;
00885         };
00886 
00887         /**
00888          * @brief represents the unreachable end of an infinite sequence
00889          */
00890         struct Unreachable {};
00891 
00892         /**
00893          * Signum function
00894          * @tparam T arbitrary scalar type
00895          * @param val function argument
00896          * @return +1 if val >= 0, -1 else
00897          */
00898         template<typename T>
00899         constexpr T sgn(T val) noexcept {
00900             return val < 0 ? T(-1) : T(1);
00901         }
00902 
00903         /**
00904          * @brief Iterator of an infinite sequence of numbers. Simply increments an internal counter
00905          * @tparam Type of the counter (most of the time this is ```std::size_t"')
00906          */
00907         template<typename T = std::size_t>
00908         struct CounterIterator : public SynthesizedOperators<CounterIterator<T» {
00909             using value_type = T;
00910             using reference = T;
00911             using pointer = void;
00912             using iterator_category = std::random_access_iterator_tag;
00913             using difference_type = std::ptrdiff_t;
00914             static_assert(std::is_integral_v<T> && !std::is_floating_point_v<T>);
00915 
00916             using SynthesizedOperators<CounterIterator<T»::operator++;
00917             using SynthesizedOperators<CounterIterator<T»::operator--;
00918 
00919             /**
00920              * CTor.
00921              * @param begin start of number sequence
00922              * @param increment step size (default is 1)
00923              * @note Depending on the template type T, increment can also be negative.
00924              */
00925             explicit constexpr CounterIterator(T begin, T increment = T(1)) noexcept:
00926                     counter(begin), increment(increment) {}
00927 
00928             constexpr CounterIterator() noexcept: CounterIterator(T(0)) {}
00929 
00930             /**
00931              * Increments value by increment
00932              * @return reference to this
00933              */
00934             constexpr CounterIterator &operator++() noexcept {
00935                 counter += increment;
00936                 return *this;
00937             }
00938 
00939             /**
00940              * Decrements value by increment
```

```
00941             * @return reference to this
00942             */
00943            constexpr CounterIterator &operator--() noexcept {
00944                counter -= increment;
00945                return *this;
00946            }
00947
00948            /**
00949             * Compound assignment increment. Increments value by n times increment
00950             * @param n number of steps
00951             * @return reference to this
00952             */
00953            constexpr CounterIterator &operator+=(difference_type n) noexcept {
00954                counter += n * increment;
00955                return *this;
00956            }
00957
00958            /**
00959             * Compound assignment decrement. Increments value by n times increment
00960             * @param n number of steps
00961             * @return reference to this
00962             */
00963            constexpr CounterIterator &operator-=(difference_type n) noexcept {
00964                counter -= n * increment;
00965                return *this;
00966            }
00967
00968            /**
00969             * Difference between two CounterIterators
00970             * @param other right hand side
00971             * @return integer ```n"' with the smallest possible absolute value such that ```other + n
    <= *this"'
00972             * @note When other has the same increment as ```*this"', then the returned value is
    guaranteed to
00973             * fulfil ```other + n == *this"'. In the following example, this is not the case:
00974             * ```
00975             * CounterIterator a(8, 1);
00976             * CounterIterator b(4, 3);
00977             * auto diff = a - b; // yields 1 since b + 1 <= a
00978             * ```
00979             */
00980            constexpr difference_type operator-(const CounterIterator &other) const noexcept {
00981                return static_cast<difference_type>((counter - other.counter) / other.increment);
00982            }
00983
00984            /**
00985             * Equality comparison.
00986             * @param other right hand side
00987             * @return true if counter of left and right hand side are equal
00988             */
00989            constexpr bool operator==(const CounterIterator &other) const noexcept {
00990                return counter == other.counter;
00991            }
00992
00993            /**
00994             * Equality comparison with Unreachable sentinel
00995             * @return false
00996             */
00997            friend constexpr bool operator==(const CounterIterator &, Unreachable) noexcept {
00998                return false;
00999            }
01000
01001            /**
01002             * @copydoc operator==(const CounterIterator &, Unreachable)
01003             */
01004            friend constexpr bool operator==(Unreachable, const CounterIterator &) noexcept {
01005                return false;
01006            }
01007
01008            friend constexpr bool operator!=(Unreachable, const CounterIterator &) noexcept {
01009                return true;
01010            }
01011
01012            /**
01013             * Less comparison of internal counters with respect to increment of this instance
01014             * @param other right hand side
01015             * @return true if
01016             * ```
01017             * sgn(increment) **this < *other sgn(increment)
01018             * ```
01019             * where ```sgn"' is the signum
01020             * function
01021             * @note If increment is negative then both sides of the inequality are multiplied with
    -1.
01022             * For example: let ```it1 = 5"' and ```it2 = -2"' be two CounterIterators where ```it1"'
    has negative
01023             * increment. Then ```it1 < it2"' is true.
```

```
01024             */
01025            constexpr bool operator<(const CounterIterator &other) const noexcept {
01026                return sgn(increment) *  counter < sgn(increment) * other.counter;
01027            }
01028
01029            /**
01030             * Greater comparison of internal counters with respect to increment of this instance
01031             * @param other right hand side
01032             * @return true if
01033             * ```
01034             * sgn(increment) **this > *other sgn(increment)
01035             * ```
01036             * where ```sgn"' is the signum
01037             * @note If increment is negative then both sides of the inequality are multiplied with
      -1.
01038             * For example: let ```it1 = 5"' and ```it2 = -2"' be two CounterIterators where ```it1"'
      has negative
01039             * increment. Then ```it1 > it2"' is false.
01040             */
01041            constexpr bool operator>(const CounterIterator &other) const noexcept {
01042                return sgn(increment) * counter > sgn(increment) * other.counter;
01043            }
01044
01045            /**
01046             * Produces the counter value
01047             * @return value of internal counter
01048             */
01049            constexpr T operator*() const noexcept {
01050                return counter;
01051            }
01052
01053        private:
01054            T counter;
01055            T increment;
01056        };
01057
01058        /**
01059         * @brief Represents an infinite range of numbers
01060         * @tparam T type of number range
01061         */
01062        template<typename T = std::size_t>
01063        struct CounterRange {
01064            /**
01065             * CTor
01066             * @param start start of the range
01067             * @param increment step size
01068             * @note Depending on the template type T, increment can also be negative.
01069             */
01070            explicit constexpr CounterRange(T start = T(0), T increment = T(1)) noexcept:
01071                start(start), increment(increment) {}
01072
01073            /**
01074             * @return CounterIterator representing the beginning of the sequence
01075             */
01076            [[nodiscard]] constexpr CounterIterator<T> begin() const noexcept {
01077                return CounterIterator<T>(start, increment);
01078            }
01079
01080            /**
01081             * @return Sentinel object representing the unreachable end of the sequence
01082             */
01083            [[nodiscard]] static constexpr Unreachable end() noexcept {
01084                return Unreachable{};
01085            }
01086
01087        private:
01088            T start;
01089            T increment;
01090        };
01091    }
01092
01093    /**
01094     * Function that is used to create a impl::ZipIterator from an arbitrary number of iterators
01095     * @tparam Iterators type of iterators
01096     * @param iterators arbitrary number of iterators
01097     * @return impl::ZipIterator
01098     * @note ZipIterators have the same iterator category as the least powerful underlying operator.
      This means that
01099     * for example, zipping a random access iterator and a bidirectional iterator only yields a
      bidirectional
01100     * impl::ZipIterator
01101     * @relatesalso impl::ZipIterator
01102     */
01103    template<typename ...Iterators>
01104    constexpr auto zip_i(Iterators ...iterators) -> impl::ZipIterator<std::tuple<Iterators...>> {
01105        using IteratorTuple = std::tuple<Iterators...>;
01106        return impl::ZipIterator<IteratorTuple>(IteratorTuple(std::move(iterators)...));
```

```
01107     }
01108
01109     /**
01110      * Function that can be used in range based loops to emulate the zip iterator from python.
01111      * As in python: if the passed containers have different lengths, the container with the least
      items decides
01112      * the overall range
01113      * @tparam Iterable Container types that support iteration
01114      * @param iterable Arbitrary number of containers
01115      * @return impl::ZipView class that provides begin and end members to be used in range based
      for-loops
01116      * @relatesalso impl::ZipView
01117      */
01118     template<typename ...Iterable>
01119     constexpr auto zip(Iterable &&...iterable) {
01120         return impl::ZipView<Iterable...>(std::forward<Iterable>(iterable)...);
01121     }
01122
01123     /**
01124      * Zip variant that does not allow manipulation of the container elements
01125      *
01126      * @copydoc zip
01127      */
01128     template<typename ...Iterable>
01129     constexpr auto const_zip(Iterable &&...iterable) {
01130         return impl::ZipView<impl::traits::reference_if_t<std::is_lvalue_reference_v<Iterable>,
01131
      std::add_const_t<std::remove_reference_t<Iterable»>...>(std::forward<Iterable>(iterable)...);
01132     }
01133
01134     namespace impl{
01135         template<typename TZip, typename ...Iterable>
01136         constexpr auto zip_enumerate_impl(TZip &&tZip, Iterable &&...iterable) {
01137             if constexpr (sizeof...(Iterable) == 0 ||
      !(std::is_integral_v<std::remove_reference_t<Iterable» || ...)) {
01138                 return std::forward<TZip>(tZip)(impl::CounterRange(0ul, 1ul),
      std::forward<Iterable>(iterable)...);
01139             } else {
01140                 constexpr auto CtrRangeArgsIdx = impl::index_of<std::is_integral, 0,
01141
      std::remove_reference_t<Iterable>...>();
01142                 static_assert(CtrRangeArgsIdx >= sizeof...(Iterable) - 2);
01143                 auto [its, enumArgs] = impl::tuple_split<CtrRangeArgsIdx>(
01144                         std::forward_as_tuple(std::forward<Iterable>(iterable)...));
01145                 return std::apply(std::forward<TZip>(tZip), std::tuple_cat(std::make_tuple(
01146                         std::make_from_tuple<impl::CounterRange<
01147                                 std::remove_reference_t<std::tuple_element_t<0, decltype(enumArgs)»»(
01148                                 enumArgs)), std::move(its)));
01149             }
01150         }
01151     }
01152
01153     /**
01154      * Function that can be used in range based loops to emulate the enumerate iterator from python.
01155      * @tparam Container Container type that supports iteration
01156      * @tparam T type of enumerate counter (default std::size_t)
01157      * @param container Source container
01158      * @param start Optional index offset (default 0)
01159      * @param increment Optional index increment (default 1)
01160      * @return impl::ZipView that provides begin and end members to be used in range based for-loops.
01161      * @relatesalso impl::ZipView
01162      */
01163     template<typename Container, typename T = std::size_t>
01164     constexpr auto enumerate(Container &&container, T start = T(0), T increment = T(1)) {
01165         return zip(impl::CounterRange(start, increment), std::forward<Container>(container));
01166     }
01167
01168     /**
01169      * enumerate variant that does not allow manipulation of the container elements
01170      *
01171      * @copydoc enumerate
01172      */
01173     template<typename Container, typename T = std::size_t>
01174     constexpr auto const_enumerate(Container &&container, T start = T(0), T increment = T(1)) {
01175         return const_zip(impl::CounterRange(start, increment), std::forward<Container>(container));
01176     }
01177
01178     /**
01179      * combination of zip and enumerate, i.e. returns an impl::ZipView that contains an enumerator at
      the first position
01180      * @tparam Iterable Types of arguments
01181      * @param iterable arbitrary number of iterables followed by optionally a start and an increment
01182      * @return impl::ZipView with prepended enumerator
01183      */
01184     template<typename ...Iterable>
01185     constexpr auto zip_enumerate(Iterable &&...iterable) {
01186         return impl::zip_enumerate_impl([](auto &&... args) { return
```

```
      zip(std::forward<decltype(args)>(args)...); },
01187                                         std::forward<Iterable>(iterable)...);
01188     }
01189
01190     /**
01191      * zip_enumerate variant that does not allow manipulation of the container elements
01192      *
01193      * @copydoc zip_enumerate
01194      */
01195     template<typename ...Iterable>
01196     constexpr auto const_zip_enumerate(Iterable &&...iterable) {
01197         return impl::zip_enumerate_impl([](auto &&... args) { return
      const_zip(std::forward<decltype(args)>(args)...); },
01198                                         std::forward<Iterable>(iterable)...);
01199     }
01200
01201 }
01202
01203 namespace std {
01204
01205     template<typename ...Ts>
01206     struct tuple_size<iterators::impl::RefTuple < Ts...» : integral_constant<std::size_t,
      sizeof...(Ts)> {};
01207
01208     template<std::size_t Idx, typename ...Ts>
01209     struct tuple_element<Idx, iterators::impl::RefTuple<Ts...» {
01210         using type = std::tuple_element_t<Idx, std::tuple<Ts...»;
01211     };
01212 }
01213
01214 #endif //ITERATORTOOLS_ITERATORS_HPP
```

Index

ALL_NOEXCEPT
    Iterators.hpp, 41

begin
    iterators::impl::CounterRange< T >, 20
BINARY_TUPLE_FOR_EACH
    Iterators.hpp, 41

const_enumerate
    iterators, 3
const_zip
    iterators, 4
const_zip_enumerate
    iterators, 4
CounterIterator
    iterators::impl::CounterIterator< T >, 13
CounterRange
    iterators::impl::CounterRange< T >, 20

DERIVE_VIEW_INTERFACE
    iterators::impl, 8

end
    iterators::impl::CounterRange< T >, 21
enumerate
    iterators, 5

getIterators
    iterators::impl::ZipIterator< Iterators >, 31

iterators, 2
    const_enumerate, 3
    const_zip, 4
    const_zip_enumerate, 4
    enumerate, 5
    zip, 5
    zip_enumerate, 6
    zip_i, 7
Iterators.hpp, 38
    ALL_NOEXCEPT, 41
    BINARY_TUPLE_FOR_EACH, 41
    TYPE_MAP, 41
    TYPE_MAP_ALIAS, 41
    TYPE_MAP_DEFAULT, 42
iterators::impl, 7
    DERIVE_VIEW_INTERFACE, 8
    sgn, 10
iterators::impl::CounterIterator< T >, 11
    CounterIterator, 13
    operator!=, 13
    operator<, 16
    operator<=, 16
    operator>, 17
    operator>=, 18
    operator++, 14
    operator+=, 14
    operator-, 14
    operator--, 15
    operator-=, 16
    operator==, 17, 19
    operator[], 18, 19
    operator*, 13
iterators::impl::CounterRange< T >, 19
    begin, 20
    CounterRange, 20
    end, 21
iterators::impl::RefTuple< Ts >, 21
iterators::impl::SynthesizedOperators< Impl >, 22
    operator!=, 24
    operator<=, 25
    operator>=, 25
    operator+, 27
    operator++, 24
    operator-, 28
    operator--, 25
    operator[], 26
iterators::impl::traits, 10
iterators::impl::Unreachable, 28
iterators::impl::ZipIterator< Iterators >, 29
    getIterators, 31
    operator!=, 31
    operator<, 35
    operator<=, 35
    operator>, 36
    operator>=, 37
    operator++, 32
    operator+=, 32
    operator-, 33
    operator--, 33, 34
    operator-=, 34
    operator==, 36
    operator[], 37
    operator*, 31
    zip_i, 38

operator!=
    iterators::impl::CounterIterator< T >, 13
    iterators::impl::SynthesizedOperators< Impl >, 24
    iterators::impl::ZipIterator< Iterators >, 31
operator<
    iterators::impl::CounterIterator< T >, 16
    iterators::impl::ZipIterator< Iterators >, 35
operator<=
    iterators::impl::CounterIterator< T >, 16
    iterators::impl::SynthesizedOperators< Impl >, 25
    iterators::impl::ZipIterator< Iterators >, 35
operator>
    iterators::impl::CounterIterator< T >, 17
    iterators::impl::ZipIterator< Iterators >, 36
operator>=
    iterators::impl::CounterIterator< T >, 18
    iterators::impl::SynthesizedOperators< Impl >, 25

iterators::impl::ZipIterator< Iterators >, 37
operator+
iterators::impl::SynthesizedOperators< Impl >, 27
operator++
iterators::impl::CounterIterator< T >, 14
iterators::impl::SynthesizedOperators< Impl >, 24
iterators::impl::ZipIterator< Iterators >, 32
operator+=
iterators::impl::CounterIterator< T >, 14
iterators::impl::ZipIterator< Iterators >, 32
operator-
iterators::impl::CounterIterator< T >, 14
iterators::impl::SynthesizedOperators< Impl >, 28
iterators::impl::ZipIterator< Iterators >, 33
operator--
iterators::impl::CounterIterator< T >, 15
iterators::impl::SynthesizedOperators< Impl >, 25
iterators::impl::ZipIterator< Iterators >, 33, 34
operator-=
iterators::impl::CounterIterator< T >, 16
iterators::impl::ZipIterator< Iterators >, 34
operator==
iterators::impl::CounterIterator< T >, 17, 19
iterators::impl::ZipIterator< Iterators >, 36
operator[]
iterators::impl::CounterIterator< T >, 18, 19
iterators::impl::SynthesizedOperators< Impl >, 26
iterators::impl::ZipIterator< Iterators >, 37
operator∗
iterators::impl::CounterIterator< T >, 13
iterators::impl::ZipIterator< Iterators >, 31

sgn
iterators::impl, 10

TYPE_MAP
Iterators.hpp, 41
TYPE_MAP_ALIAS
Iterators.hpp, 41
TYPE_MAP_DEFAULT
Iterators.hpp, 42

zip
iterators, 5
zip and enumerate iterators, 1
zip_enumerate
iterators, 6
zip_i
iterators, 7
iterators::impl::ZipIterator< Iterators >, 38